"ALEXANDRU-IOAN CUZA" UNIVERSITY OF IAȘI

**FACULTY OF COMPUTER SCIENCE**

BACHELOR'S THESIS

# FerrumOS - An Operating System in Rust

proposed by

**Marian-Alexandru Pârlea**

**Session:** July, 2025

Supervisor

**Lect. Dr. Cristian Vidrașcu**

”ALEXANDRU-IOAN CUZA” UNIVERSITY OF IAȘI

**FACULTY OF COMPUTER SCIENCE**

# FerrumOS - An Operating System in Rust

**Marian-Alexandru Pârlea**

**Session:** July, 2025

Supervisor

**Lect. Dr. Cristian Vidrașcu**

# Contents

# Motivation

"Any sufficiently advanced technology is indistinguishable from magic" [3]

Over the college years, I've always been interested in the low-level side of programming. I have gone through many lectures that have profoundly influenced me throughout this journey, most of them based on the connection between the software world that many of us interact with every day and the hardware that many see as a black box that just works and lets us do some awesome stuff with our skills. After all those courses, that was the moment when I realized that I did not start at a technology-based college just because I like what is new or what is trending; I started studying at this college because I am curious. I am curious about how things work together. I wanted to know what the magic is that makes some silicon pieces fascinating, what makes them able to store that much information, or how they are making operations faster than any human on earth.

In the moment that I was given the chance to choose a project that would represent my whole progress throughout my university studies, I knew what I needed to choose. I wanted to know how this magic works. How does the hardware know how to run my Tetris game written in an English-like language? So I chose the one thing that was the closest to that magic that I know, the effective communication method between our programs–written in feature-rich languages–and the circuits hidden within the PCBs of those black boxes sitting on our desks, which deliver so much power to us. We need an operating system.

Writing an operating system is no easy task, especially at the level of expertise a second-year computer science student possesses. To succeed in building your project, you need solid knowledge of data structures, algorithms, and operating system internals. You must understand concepts like memory allocators, paging, byte-level read-/write operations, inter-process communication protocols, interrupt handling, and much more. Consequently, completing a fully functional operating system is a mon-

umental task that requires immense effort to accomplish. Throughout my journey, I encountered countless obstacles, from errors appearing out of nowhere to standards that were poor or even not documented at all, forcing me to search through old or obscure pages in search of an answer.

System development is not for everyone. Some will try and quit; others will create masterpieces. Without exception, everyone learns from the process. Having endured these challenges to complete my project, I realized that the OSDev community needs a simpler way to access essential information–without spending hours scouring obscure pages for a solution. This is why I've dedicated my bachelor's thesis to making operating system development more accessible.

# Introduction

A modern computer consists of one or more processors, main memory, storage devices, printers, mice, keyboards, monitors, network interfaces, and various other input/output devices–all forming an extremely complex computing system. Assuming we want to write a program for such a system, we would need to be familiar with all the other underlying concepts behind each individual component's operation–a monumental task in itself, not to mention the need to efficiently utilize those devices to develop even the most basic applications. Such an arrangement creates the need for an abstraction layer that handles all device communication details, provides efficiency and security, and, most importantly, offers a way to develop applications without requiring the developer to know every operational detail of the hardware being used.

Thus emerged the need for an operating system–a piece of software that ensures smooth communication between various computer devices, guarantees all operations are performed in a secure and efficient manner, and, most crucially, provides a clear foundation upon which all the applications we use today can be built.

An operating system serves two primary functions: providing developers with a clean and abstract set of resources that hide low-level complexity induced by the hardware and managing the system's available resources. Opinions are divided, and when hearing someone talking about operating systems, they would mainly focus on one of two roles that an operating system fulfills; therefore, in what follows, we will present a brief analysis of each. [23]

## The Operating System as an Abstraction to the Hardware

The primitivity level that is present in the architecture (instruction sets, memory management, input/output operations, communication methods) of many computers presents significant challenges for developers who have to develop pieces of software on those systems. A good example would be the SATA standard, documented

in 2007[4], which describes in over 450 pages how one would interact with a storage medium to fulfill certain operations—this standard has since developed and grown in size and complexity. Certainly no programmer would want to read and understand hundreds of pages of documentation for each different device that is used in his project to do common tasks like manipulating an image or writing a report that has to be sent over the internet. This is the reason why there are pieces of software specially built for those use cases: drivers.

Drivers are pieces of code that ensure an abstraction level over the underlying complexity of each and every device that can be connected to a computer. Therefore, make sure that anyone who wants to build an application for a certain use case does not necessarily need to know how the devices they use work, avoiding unnecessary complexity and making the codebase more intelligible, safe, and fast.

## The Operation System as a Resource Manager

While describing a system in a top-down manner, we can say that its whole purpose is to provide an abstraction level over the complexity provided by the hardware. If we change the perspective and we rather look in a bottom-up manner, we can make the affirmation, without making any mistake, that an operating system has the important role of managing all resources provided by the machine and making sure that they work correctly and in relation with one another. But what does it mean for resources to work in a beneficial relationship with one another? Let's take an example to understand this concept better. Let's say that we have one printer and three applications that all want to print something. Without any management from the operating system and the programs running in parallel, the piece of paper that would be printed by the printer would have some `a` from program 1, some `b` from program 2, and some `c` from program 3, and subsequently would look something like this: `aababbbaccabbaccccabccca`. And no one that would need to print something would want the paper to be corrupted by other programs just because the operating system was unable to manage them.

Therefore, the operating system has to manage some sort of multiplexing, letting applications use certain resources in a controlled manner. There are two main ways to implement multiplexing: across time and across space.

Multiplexing across time works when there is a certain resource that we do not

have enough of for every application that has to run on our system, for example, the CPU or the printer in the previous example. The way that the operating system manages those kinds of resources is by giving each process a predefined time in which it can freely use the needed resource; after that time expires, the next application gets to use the resource, and so on. In our example with the printer, the operating system lets program 1 use the printer and waits a period of time until this one finishes its work with the printer, then program 2 gets access, then program 3. Therefore, the operating system ensures that no more than one process utilizes this type of resource at any given moment.

Space multiplexing works when applications running on the machine only need a small part of the resource. One example is loading programs into the main memory to schedule them to a resource that needs time multiplexing. Because the main memory has a huge capacity compared with the required space needed by a process, the operating system has to dispatch areas of that memory such that every process that wants to run has its own region where it can be loaded. Another example, which is more general, is to imagine a central computer in a company; the employees of that company access it to write and read documents. In this case, the operating system has to assign to each employee a portion of the storage device and manage its access in order to not interfere with the work of the other employees.

## What is the aim?

Considering all the statements mentioned above, I consider that having an operating system as an abstraction layer between the hardware and the applications that we use in our day-to-day lives, along with the need to manage all the resources that the machine provides, is essential in terms of efficiency, security, and ease of development and usage. Therefore, the abstraction over hardware and resource management are some of the most important problems in the computer science field that need our constant attention. Thus, this paper will explore the inner workings of an operating system, from implementation details to security considerations, aiming to provide a comprehensive understanding of how an operating system works and what it means to build your own. We will analyze core components such as drivers, memory management, concurrency, interrupts, file systems, and many more. We will talk about common pitfalls in developing core components, and we will analyze some design

trade-offs between efficiency and usability—all while adhering to Rust's security guarantees and memory-safe model.

# Chapter 1

# Graphics Foundations

When thinking about a computer, in most cases we think about a central unit–which houses all components from Central Processing Units (CPUs) to storage mediums and Random Access Memory (RAM)—and a modality to interact with it, minimally a keyboard and some sort of display that would give feedback to the given input. Over the years, the technology present in computer displays has evolved from cathode-ray tube to liquid-crystal display and organic light-emitting diode. Those evolutions in display technology required certain decisions to be made about the communication between the graphics card and the display. In this chapter we will talk about two important standards that deal with the communication protocol: Video Graphics Array (VGA) and Framebuffer.

## 1.1 VGA

VGA was introduced by IBM in 1987 for its PS/2 line of PCs. The original chipset was able to display up to 16 colors at a screen resolution of 640 x 480 pixels, and while by today's standards this resolution is considered low, at the time it was sufficient to display text and render basic graphics. [6][12]

Although the VGA standard is capable of rendering basic graphics through its registers, in this section we are going to focus on its text mode, which will give us the capability to write text to the screen directly without worrying about parsing font files or writing our own bitmap for the characters.

The VGA text mode is a simple way to print text. Rather than working with each pixel at a time, this VGA mode splits the screen into rows and columns and stores it

as a two-dimensional array, which typically has 25 rows and 80 columns. Each cell present in the buffer describes a single screen character and has the format presented in Figure 1.1.

| Offset | Size | Value |
|---|---|---|
| 0 | 8 | CCSID 437 code point |
| 8 | 4 | Foreground color |
| 12 | 3 | Background color |
| 15 | 1 | Blink |

Figure 1.1: Entry format

The first byte represents the character's code in CCSID 437, or more commonly known as Code Page 437, which is a subset of printable ASCII characters that include English lowercase and uppercase letters, some diacritics, Greek letters, icons, and line-drawing symbols, as stated in the original specification [7]. The second byte defines how the character is being displayed; the first four bits represent the foreground color, the next three the background color, and the final bit shows whether the character should blink or not. In Figure 1.2 are presented the available color codes for VGA text mode.

| Value | Color | Value | Color |
|---|---|---|---|
| 0x0 | Black | 0x8 | Dark Grey |
| 0x1 | Blue | 0x9 | Light Blue |
| 0x2 | Green | 0xA | Light Green |
| 0x3 | Cyan | 0xB | Light Cyan |
| 0x4 | Red | 0xC | Light Red |
| 0x5 | Magenta | 0xD | Pink |
| 0x6 | Brown | 0xE | Yellow |
| 0x7 | Light Gray | 0xF | White |

Figure 1.2: Available colors

Now, in order to get something written on the screen, we need to write to a memory-mapped I/O address (i.e., `0xB8000`). This means that the reads and writes happen directly on the VGA frame buffer, therefore changing what gets shown on the

screen. In order to write something to the screen, you need to follow the following steps.

1. Initialize your text as a byte array.
2. Get a pointer to the start of the VGA buffer (`0xB8000`).
3. Select a background and a foreground color.
4. Compute the value you need to write to the buffer as

$$value = (byte << 8) \ \& \ ((background << 4) \mid foreground)$$

5. Write the corresponding value to the VGA buffer.

Now there is the question: why do we need any other way to facilitate communication if this one is so easy to understand and use? Well, there are some reasons:

- Limited Graphics Capabilities: VGA text mode only allows for character-based output and a limited set of colors, making it unsuitable for modern graphical user interfaces or complex graphics rendering.
- Resolution Constraints: The standard VGA text mode is restricted to low resolution displays, therefore unable to take advantage of the modern displays.
- Direct Hardware Access: Writing directly to memory-mapped I/O can be unsafe and is not portable across different hardware platforms.

## 1.2 Framebuffer

Because of its limited 16-color 640x480 resolution [31], many modern bootloaders choose to not include support for the old VGA standard and instead opt in for a linear framebuffer that can support the display's native resolution and full range of colors, going from 480p to 1080p, 4k, and even higher. Despite the linear framebuffer being able to take advantage of the immense range of colors and resolutions, this flexibility comes at the cost of some complexity; in order to print anything on the screen, the driver developer has to build methods for primitive shapes and especially for text. Therefore, to continue with the specifications for the framebuffer, we are going to briefly present some methods of how to print text on the screen.

### 1.2.1 Fonts

Once using the framebuffer, you no longer have the BIOS or the hardware to draw fonts for you. So you need to write your own methods to parse font files in order to see some text on the screen. There are many well-known font formats, such as TrueTypeFonts (TTFs) or OpenTypeFonts (OTFs), but those are mainly vector fonts [24], and as shown in Sebastian Lague's video about text rendering [11], the complexity of this topic would make it good enough to be a project on its own. Because it's not the purpose of this paper to explain font specifications, we are going to dive into a significantly simpler format: PC Screen Font (PSF)

Commonly known as PSF or PSFU, those are a set of fonts that the majority of Linux distributions come with by default–so if you are on a Linux distribution, you can check them out by going into `/usr/share/kbd/consolefonts`. PSF fonts are significantly easier to work with because of how the memory representation of a character looks–the way that a character is represented in memory is called a *glyph*, therefore in what follows, we are going to use the term *glyph* to refer to the memory representation of a character–making a trade-off between expressiveness and simplicity.

#### 1.2.1.1 The Header

When parsing a byte file, in this case a PSF font, we will stumble upon its header. The header of a file contains common information about its content. The PSF currently has two versions for its header–the memory layout is presented in Figure 1.3 and Figure 1.4–therefore being able to add or to remove certain features.

In order to know which version of the header we are working with, we have to check the first four bytes. If we locate `36 04` in the first two, that means we are working with a version 1 PSF header, and we are limited to the information presented in Figure 1.3. The version 1 header provides limited information, specifically regarding which modes are active and the size of the glyph.

Given the presented modes, we can determine whether we have 512 glyphs (if `0x01` is set) or 256 and if we have a Unicode table (if `0x02` or `0x04` is set). The Glyph size field in the header will always refer to the height in pixels because the width of a character is always 8 pixels. [2]

If the magic number does not start with `36 04`, we have to check if it is `86 4A B5 72`; if the number checks, then we are working with a version 2 PSF. In this version,

| Offset | Size | Description |
|---|---|---|
| 0 | 2 | Magic bytes |
| 2 | 1 | Font Mode |
| 3 | 1 | Glyph size |

Figure 1.3: PSF1 type header

the Font mode field from version 1 was divided into two separate fields: *Length*, which tells us how many glyphs there are, and *Flags*, where if `0x01` is set, that means that we have a Unicode table immediately after the glyphs table.

The other fields hold general information such as the width (*Width*) and the height (*Height*) of a glyph together with how many bytes are needed for each character (*Glyph size*). The *Version* field tells us the minor version of the header (usually 0), and the *Header size* field tells us the size of the header (usually 32) and where the glyph table starts. [2]

| Offset | Size | Description |
|---|---|---|
| 0 | 4 | Magic bytes |
| 4 | 4 | Version |
| 8 | 4 | Header size |
| 12 | 4 | Flags |
| 16 | 4 | Length |
| 20 | 4 | Glyph size |
| 24 | 4 | Height |
| 28 | 4 | Width |

Figure 1.4: PSF2 type header

#### 1.2.1.2 The Glyphs

As stated above, a glyph represents how a character is stored in memory. In order to know how many bytes we need to read from the file, we need to multiply the glyph count by the size of a glyph. For example, if the font has 512 characters, and each glyph has a width of 1 byte and a height of 16 bytes, then we would need to read $512 \cdot 1 \cdot 16 = 8192$ bytes.

Now that we know how many bytes we need to read, we can start displaying our font by lighting up each pixel that corresponds to a 1 and turning off each pixel corresponding to a 0. Therefore we can see how each character in our font looks like. But how do we know what code each one has and how we can write a print function that can use the font? Regarding the codes for each character, if in the header it is stated that there is no Unicode table, then the first glyph corresponds to the character with code 0, the second to the character with code 1, and so on. The case when we actually have a Unicode table is described in the following section. Regarding the translation between your string in your application and an actual glyph from the loaded font, check subsubsection 1.2.1.4.

#### 1.2.1.3 Unicode table

If the PSF header indicates that there exists a Unicode table, this table will be found after the glyph table, and every glyph present in the font will have an entry here. Therefore, in order to print a character, you will need to find its Unicode value in the table and use that index to print the glyph. In Figure 1.5 is presented the general form of the glyph unicode entry, where `<uc>` represents two bytes of information and $\star$ indicates 0 or more occurrences of the previous item.

```
<unicodedescription> := <uc>*<seq>*<term>
<seq> := <ss><uc><uc>*
<ss> := psf1 ? 0xFFFE : 0xFE
<term> := psf1 ? 0xFFFF : 0xFF
```

Figure 1.5: PSF glyph unicode entry[2]

For example, let's say we read the following bytes: `00C5`, `212B`, `FFFE`, `0041`, `030A`, `FFFF`[2]. As per the regular expression presented in Figure 1.5, we can identify two Unicode sequences [`00C5`, `212B`] and [`0041`, `030A`] separated by `FFFE` (`FE` for version 2) and terminated by `FFFF` (`FF` in version 2). If the character that we are trying to print has this Unicode value, then we have found the correct glyph. Now the question that appears is, how do I obtain the glyph if my language uses UTF-8 to encode its strings? This question is answered in the following section.

#### 1.2.1.4 UTF-8 to code-point

In some modern programming languages, like Rust, strings are represented in the UTF-8 format, but in order to print something to the screen, we need to parse the string character by character, and as per the official documentation[22], a char is a Unicode scalar value, which can be interpreted as the decoded version of a UTF-8 character.

In order to easily convert any UTF-8 value to its code-point counterpart, check out Figure 1.6, where it is explained what the starting of each byte means. After checking that the starting values are correct, you need to remove them and rebuild the value.

| Starting pattern | Meaning |
|---|---|
| `0b0` | 1 byte UTF-8 |
| `0b110` | 2 byte UTF-8 |
| `0b1110` | 3 byte UTF-8 |
| `0b11110` | 4 byte UTF-8 |
| `0b10` | 2nd, 3rd, 4th byte |

Figure 1.6: Starting bits in an UTF-8 encoding[10]

For example, let's say that you want to write the text corresponding with the following byte order: `F0 9F 8D 8E 2B CF 80 21`. Converting these bytes into binary and checking the patterns in Figure 1.6, we can divide the byte sequence into the following groups: [`11110000 10011111 10001101 10001110`], [`00101011`], [`11001111 10000000`], [`00100001`]. Let's take the first group `11110`000 `10`011111 `10`001101 `10`001110. After removing the underlined bits, we get `0 0001 1111 0011 0100 1110`, which is `U+01F34E`. Doing the same thing to the other groups, we will obtain `U+002B U+03C0 U+0021`, and therefore the Unicode code points for each character.

### 1.2.2 The linear framebuffer

The first thing that we need to do in order to have access to the linear framebuffer is to request it from our bootloader. Different bootloaders have different protocols for resource retrieval. Since this paper is focused on building an operating system in Rust, the easiest way is to use the Limine bootloader. The Limine bootloader provides the user with a Rust crate–a Rust crate is some sort of package that we can import into our project and use–that provides a simple API that allows us to request information or

resources from the bootloader. It is to be noted that Rust is not the only programming language that Limine can work with; there are multiple other languages that come with libraries that can work with Limine (e.g., C/C++, Zig). The following steps are made using the Limine boot protocol [14]. If other protocols are used, minor changes can be made in order to get things to work, but those are not presented in this paper.

In order to get access to the linear framebuffer, you need to make a framebuffer request to the bootloader. The bootloader will give you a response. In order to build a safe and efficient operating system, you need to make sure that the responses received from the bootloader contain actual data that you can work with and not just assume that what you get is usable. According to the Limine crate documentation, after making a request for the framebuffers, you will get a response containing the revision of the request and a list of framebuffer objects. For each framebuffer, you get a list of properties, the most important presented in Figure 1.7

| Field | Description |
|---|---|
| `addr` | The address of the framebuffer |
| `width` | The width of the framebuffer |
| `height` | The height of the framebuffer |
| `pitch` | The distance between rows in bytes |
| `bpp` | The number of bits per pixel |

Figure 1.7: Framebuffer information available[13]

It is to be noted that the address on which the framebuffer is located has no synchronization mechanism to protect it; therefore, the user must ensure to synchronize every access to it in order to avoid undefined behavior. The pitch is not always equal to $\frac{width \cdot bpp}{8}$ because there may be bytes that act as padding in order to achieve a certain alignment.

After all information is retrieved, we need to determine the row and the column where we want to draw a pixel. Those can be determined by using the following formulas: $row = y \cdot pithc\ column = x \cdot bpp$. Keep in mind that $x$ and $y$ must be in the screen resolution. Now, in order to get the address on which we need to write in order to draw the pixel, we need to add to the base address (`addr`) the column and the row previously calculated.

Nowadays most framebuffers use the RGB format in order to color a pixel. If

the `type` of the framebuffer is set to RGB, then there should be attributes that indicate where each color starts on (or how many bits to shift in order to get there) and the size of each color in bits.

# Chapter 2

# Interrupts and CPU Exception

An interrupt is the method used by the CPU to communicate to our system that something unexpected or unpredictable has happened and that it needs to be handled immediately. When an interrupt occurs, the CPU will treat the interrupt by calling a function commonly referred to as an interrupt handler. The interrupt handler, as the name implies, has to handle the interrupt in a safe and efficient manner.

Since the operating system this paper is based on is implemented for an `x86` architecture, there are a few things worth mentioning. The `x86` architecture makes a clear distinction between hardware and software interrupts. Therefore, define a software interrupt as an interrupt that occurs after the instruction `int` has been used or the CPU noticed something undesirable about the execution (e.g., accessing unpermitted memory) and is commonly referred to as *CPU Exception*. A detail about the hardware interrupts is that sometimes, there can be an error code that the user has to take into account when writing the handler for that specific interrupt.

Interrupts acquire their name because they interrupt the normal flow of execution, stop whatever runs on the CPU, execute the handler function, and then resume to the previously running code. Since there are moments when stopping the current flow of execution can cause the system to panic or cause undefined behavior, we can tell the CPU that on a certain portion of code, we do not want any interrupt to stop the current execution. This capability is possible through two assembly instructions:

- `cli` – Clears the interrupt flag, therefore preventing the CPU from serving interrupts
- `sti` – Sets the interrupt flag, therefore allowing the CPU to serve interrupts.

There is one thing to note about this behavior. Clearing the interrupt flags does not stop all interrupts from happening. There is a category of interrupts, Non Maskable Interrupts (`NMI`), that usually occur when there is a critical hardware failure and the system cannot continue with its normal flow; therefore, a system panic is in most cases the way to go, letting the user know that something wrong happened and a restart is required. Those interrupts are extremely rare, having low odds of happening, but you need to be aware of them and try to handle them properly.

In what follows, we are going to analyze in more detail how each interrupt category (i.e., CPU exceptions, hardware interrupts) works and what is needed for its correct handling.

## 2.1 CPU Exceptions

Working on an `x86` architecture provides you about 20 interrupts that you have to take into account. While most of them are explored in more depth in a following section (see subsection 2.1.1), the most important ones are

- **Page Fault** – A page fault occurs when trying to illegally access memory. For example, if the current instruction tries to read from an unmapped page or write to a read-only page, this exception will occur.
- **Invalid Opcode** – This typically happens when attempting to execute new instructions on outdated machines without any support in place.
- **General Protection Fault** – This exception has the broadest range of causes, which can occur due to various types of access violations, such as attempting to execute privileged instructions in user-level code or writing to reserved fields in configuration registers. [16]
- **Double Fault** – When an exception occurs, the CPU tries to call its handler; if it doesn't exist, or another exception happens exactly while the handler is running, a double fault exception is raised.
- **Triple Fault** – If an exception occurs while the CPU is calling the double fault handler, it issues a fatal triple fault. This type of exception cannot be caught, most CPUs reacting to it by resetting themselves and rebooting the operating system.

### 2.1.1 Interrupt Descriptor Table

The Interrupt Descriptor Table (IDT) is a binary data structure specific to the IA-32 and x86-64 architectures. It is similar to the Global Descriptor Table in structure (presented in subsection 2.1.3). The main objective of the IDT is to provide the CPU with a list of entries where it can determine handlers for each interrupt. In Figure 2.1 is presented how an entry looks in memory.

| Offset | Size | Description |
|---|---|---|
| 0 | 2 | Address low |
| 2 | 4 | Selector |
| 4 | 2 | Options |
| 6 | 2 | Address mid |
| 8 | 4 | Address high |
| 12 | 4 | Reserved |

Figure 2.1: IDT Entry memory layout

The fields referring to the address (i.e., Address Low, Address Mid, and Address High) represent the 64-bit address of our handler function, low representing bits `15:0`, mid `31:16`, and high `63:32`. The reserved field should always be zero and not written on. The selector refers to the code segment in the GDT structure that we are going to use. The Options field is represented by a bitfield structure (presented in Figure 2.2) where

- **P** – If this is zero, that means this descriptor is not valid and we do not support handling this vector.
- **DLP** – The descriptor privilege level determines the highest CPU ring that can trigger this interrupt via software.
- **T** – In long mode, there are two types of descriptors that we can put in an entry: traps and gates.
- **ISTI** – If set to zero, do not switch stacks. For any other value, switch to the n-th stack in the Interrupt Stack Table when the handler is called. The IST will be discussed in more depth in subsection 2.1.2.

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 | 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| P | DLP | | 0 | 1 | | | T | Reserved | | | | | ISTI | | |

Figure 2.2: IDT Entry Options memory layout

Each exception has a predefined IDT index. For example, the invalid opcode exception has table index 6, while the page fault has table index 14. Indexes, types, and error codes are presented for all exceptions in Appendix A. Knowing each index for each exception, the hardware can quickly retrieve the entry corresponding with each interrupt.

#### 2.1.1.1 The handler function

When working on a project with the final goal of fulfilling a specific purpose within a system, we usually prefer to make the code readable; therefore, we break the codebase into classes and functions. Understanding the inner workings of a function's call is crucial. When you call a function anywhere in your code, the compiler translates it, taking into account some steps:

1. **Saving the caller's state** – The caller saves registers that must be preserved across calls (e.g., `rbx` `rbp`).
2. **Parameter passing** – Saving into registers parameters, if we have a relatively small amount of them, or pushing them onto the stack.
3. **Function invocation** – As simple as running the `call` instruction that pushes the return address (next instruction's address) onto the stack and jumps to the function's entry point.
4. **Function execution** – The function adjusts its stack and runs.
5. **Returning** – After the function has finished its work, the `ret` instruction is called that pops the return address from the stack and jumps to it.

Among many differences pointed out by Philipp Oppermann in its blog post [16], one of the most important ones is that you do not know if there would be a return address present, and if there is one, you cannot know if the return address that you took from the stack is the actual return address or the error code pushed by the interrupt [15]. Therefore, the handler must be a diverging function.

#### 2.1.1.2 Loading the IDT

Now that we've talked about how a handler should work, we need to talk about how we tell the CPU where to locate our IDT. Fortunately, there is an assembly instruction that sets up a register where the CPU searches for the IDT. The `lidt` expects two parameters:

1. **Limit** – The maximum addressable byte in the table. This value is equivalent to the table size in bytes, minus one. This argument has to be a value on two bytes (`u16`).

2. **Base** – The base address where it is expected to locate the IDT. This argument has to be a value on four bytes (`u64`).

A simple test that can be done in order to verify whether we've set up things correctly or not would be some line of assembly. If you run the instruction `int3`, a breakpoint exception would occur, and the corresponding handler should be run automatically.

### 2.1.2 Task State Segment

The Task State Segment (TSS) is a legacy structure that was normally used for hardware context switching in the 32-bit mode. Since this type of context switching is no longer supported, the TSS moved its focus toward holding information about two tables: the Privilege Stack Table and the Interrupt Stack Table. In Figure 2.3 is presented the memory layout of the TSS structure.

| Offset | Size | Description |
|---|---|---|
| 0 | 4 | reserved |
| 4 | 24 | Priviledge Stack Table |
| 28 | 8 | reserved |
| 36 | 56 | Interrupt Stack Table |
| 92 | 10 | reserved |
| 102 | 2 | I/O Map Base Address |

Figure 2.3: TSS memory layout

| First Interrupt | Second Interrupt |
|---|---|
| Divide by zero<br>Invalid TSS<br>Segment Not Present<br>Stack Segment Fault<br>General Protection Fault | Invalid TSS<br>Segment Not Present<br>Stack Segment Fault<br>General Protection Fault |
| Page fault | Page Fault<br>Invalid TSS<br>Segment Not present<br>Stack Segment Fault<br>General Protection Fault |

Figure 2.4: Causes of Double Faults

The Privilege Stack Table entry is an `[u64; 3]` array. It is usually used by the CPU when privilege changes are needed. For example, if an exception appears during the system being in user mode (Ring 3), the CPU has to switch to kernel mode (Ring 0). In this case, the CPU would switch to the 0th stack in the Privilege Stack Table.

The I/O Map Address is a field that holds an offset, from the TSS base, to the I/O Permission Bit Map. This map is checked when an interrupt needs to operate on I/O ports; therefore, if the bit is cleared (0), it can use a certain port; otherwise, not.

The interrupt stack table is an `[u64; 7]` array responsible for holding a list of clean stacks where the CPU can switch on certain interrupts. The reason why we need to change stacks is safety. In Figure 2.4 are presented the sequences of interrupts that need to happen in order to get a double fault.

A page guard is a special memory page that sits at the bottom of the stack, making it possible to detect stack overflows. This kind of page is not mapped to any physical memory; therefore, when trying to access memory that falls in this page, a page fault will occur.

What would happen if we had grown our stack so much that we were doing a stack overflow? The CPU detects the invalid access and raises a Page Fault exception. Therefore, taking the current stack frame and trying to push it to the stack in order to know what happened and what to do, but the stack is already full, and any tries to push something to it will cause another exception. Doing so, we would fall into a

triple fault exception. Therefore, for some kinds of exceptions, such as double faults, we need to move to a fresh stack in order to manage the interrupt accordingly.

In order to add a new IDT entry, we need to choose an index on which we should access the stack and to allocate an area of memory where the new stack is located. The only problem is that this is a local variable. We need to tell the CPU where to locate it. This assignment can be done using the Global Descriptor Table.

### 2.1.3 Global Descriptor Table

The main purpose of the Global Descriptor Table was to manage memory segmentation. Since paging, it lost its relevance, the only important things that it is used for are, among others, kernel/user space and TSS loading.

Entries in the GDT are of two main types: user (code and data) and system segment descriptors. Since paging, code and data descriptors no longer contain addresses, therefore spanning over the entire address space on `x86-64`, therefore, the only relevant information stored in those entries are some flags.

System descriptors, on the other hand, need a base address and a limit; therefore, the standard reserves 128 bits for these types of entries.

One thing to keep in mind when setting up the GDT is to disable (if not already) all the interrupts in order to avoid undefined behavior. After building your GDT structure in memory, we need to tell the CPU where to locate it. We can do this by running the `lgdt` instruction with our GDT's address as a parameter.

## 2.2 Hardware Interrupts

Interrupts provide a way for the attached hardware devices to notify the CPU that something happened. So instead of letting the kernel periodically poll the devices for information (e.g., checking the keyboard if a key was pressed), a better option is to do it via interrupts. Therefore, this option ensures that we do not waste time going back and forth to every device; instead, we stop execution only when we really need to. [17]

On modern PCs we have many devices connected, such as keyboards, mice, printers, gamepads, and more. Since all devices can't be connected to the CPU at once, we need something that aggregates them and alerts the CPU when needed. Such an

aggregator is usually referred to as an interrupt controller.

Most interrupt controllers are programmable, which means that they support different priority levels for different devices. For example, this allows a timer to have a greater priority than a keyboard (helpful for scheduling).

There are many different controllers. But in this paper we will talk about two of them: 8259 Programmable Interrupt Controller (PIC) and Advanced Programmable Interrupt Controller (APIC). The PIC being an older device, we will firstly talk about it, and then we will continue with APIC, which offers more functionalities–such as multi processor suport–but it is harder to set up.

### 2.2.1 PIC

The Intel 8259 is a PIC introduced in 1976. It has eight interrupt lines and several lines for CPU communication. Usually the 8259 PIC consists of two main controllers, one primary and one secondary, the secondary one connected to a port on the primary one. In Figure 2.5 is presented the structure of the two controllers, each one having an output. The output from the secondary controller goes into a port of the primary controller, and the output of the primary goes directly into the CPU.

| Secondary Interrupt Controller | | Primary Interrupt Controller | |
|---|---|---|---|
| Port | Description | Port | Description |
| 0 | Real Time Clock | 0 | Timer |
| 1 | ACPI | 1 | Keyboard |
| 2 | Available | 2 | Secondary Interrupt Controller |
| 3 | Available | 3 | Serial Port 2 |
| 4 | Mouse | 4 | Serial Port 1 |
| 5 | Co-Processor | 5 | Parallel Port 2/3 |
| 6 | Primary ATA | 6 | Floppy Disk |
| 7 | Secondary ATA | 7 | Parallel Port 1 |

Figure 2.5: Secondary Interrupt Controller

In order to access the PIC, you need to communicate with it using its ports. For the primary controller (master), we have `0x20` and `0x21` for command and data, respectively, and for the secondary controller (slave), we have `0xA0` for command and `0xA1` for data.

In order to initialize your PICs, you need to write on the command registry the command `0x11`, which translates into a request for initialization and an indication that there will be more data to be processed. One thing to be noted is that there needs to be a small delay between writes in order to give the PIC enough time to read the command. The delay is commonly achieved by writing garbage to port `0x80`, which would provide PIC enough time to read the command.

Now we need an offset that will tell the interrupts where they are starting. This offset can be any number, even zero. The only catch is that the CPU already mapped its interrupts from 0 to 31. Therefore, we can map the primary PIC to offset `32` and the secondary PIC to offset `40`, since each PIC has eight ports. This offset needs to be set on the data registry.

We need to tell the master PIC that there is a slave PIC at IRQ2, so we write `4` to its data registry and `2` to the data registry of the slave in order to let it know that it needs to fall back to the master.

Finally, we need to specify that the PICs have to use 8086 mode instead of 8080 by writing `0x1` to their data registers.

The PICs have all their interrupts masked, so there will be no interrupt from the hardware that is treated; instead, those are going to be ignored. In order to unmask them, you need to write `0x0` to the data registry of each one. If you want to be more specific, you need to set to zero the corresponding bit and write it to the registry. In order to do this, just shift `0x1` with the position of the interrupt. For example, if we want to only enable the timer, we would write to the master data registry `0xFF & !((0x1 << 0) as u8)`. Usually this action is done in a `cli sti` block in order to avoid unwanted interrupts while we set things up.

After desired interrupts are enabled, the PIC will issue an interrupt request to the CPU with the offset that we set up. For example, the timer interrupt will be signaled as Interrupt Request (IRQ) 32 if we set up the offset as 32. Therefore, there needs to be a handler for IRQ32 in the IDT in order to avoid a double fault.

Unlike CPU exceptions, hardware interrupts need to let the system know that the handling was done right and completely. Therefore, we need to write `0x20` to the command registry of the corresponding port in order to continue our execution from where we left off. The notification needs to be sent carefully; for example, if the interrupt comes from PIC2, we need to notify PIC1 and PIC2 that we have finished treating the interrupt; if it comes from PIC1, we only need to notify PIC1. Notifying

other controllers that the ones responsible for the interrupts might skip an unhappened interrupt and produce undefined behavior.

Working with PIC is easy and supports enough features that you would most likely stick with it. But what happens if you want to introduce support for multiprocessor machines? The PIC devices do not support more than one processor; therefore, you need to dig deep into the Advanced Programmable Interrupt Controller.

### 2.2.2 APIC

The Advanced Programmable Interrupt Controller, most commonly referred to as APIC, is the updated Intel standard for the older PIC. As per Intel's official documentation [9], it serves two main purposes:

- It receives interrupts from the processor's interrupt pins, from internal sources, and from an external interrupt controller. It sends these to the processor core for handling.
- In multiple processor systems, it sends and receives interprocessor interrupt messages to and from other logical processors on the system bus. Interprocessor interrupts can be used to distribute work among the processors.

Since the APIC is fairly new, some older machines may not support it. Therefore, before you start, you need to verify whether the APIC is present on your machine or not. According to the OSDev Wiki [26] [30], you need to run the `cpuid` instruction with the `eax` registry set to `0x1` in order to show CPU information. After you have executed the `cpuid` instruction, you need to examine the ninth bit of the `edx` registry, which, if set, confirms the presence of the APIC device on your machine. If this bit is not set, there is no APIC on your machine, and your system should fall back to using PIC.

The APIC device is made of two components:

- **Local APIC** – Also referred to as xAPIC, its main purpose is to manage IPIs (interprocessor interrupts) and to raise interrupts itself.
- **I/O APIC** – On personal computers there is usually only one, but there can be multiple I/O APICs on servers or industrial machines. The main purpose of this device is to manage input and output interrupts and redirect them to the xAPIC for further management.

#### 2.2.2.1 Disabling 8259 PIC

Since the APIC was a fairly new device, most computers boot up in the 8269 PIC emulation mode in order to offer backwards compatibility. Since we want to use APIC for our system, we need to correctly disable this emulation mode. In order to disable it correctly, we need to do the following:

1. **Masking** – Mask all interrupts in order to disable them in the PIC.
2. **Remapping** – You need to remap all the IRQs (this you probably already have done if you set up PIC) in order to avoid conflicts with CPU exceptions that span from 0 to 31, so you should remap them starting from at least 32.

#### 2.2.2.2 xAPIC

After correctly disabling the emulation mode that the APIC starts in, we need to gain access to it. This is done via a Model Specific Register (MSR). This instruction reads the contents of `ecx` and places the result in `eax` (low 32 bits) and `edx` (high 32 bits). In our case, the MSR that we have to access is `IA32_APIC_BASE`, which is `0x1B`. The result should have a structure like the one presented in Figure 2.6, where

- **Address** – Contains the base address of the local APIC for this processor core.
- **E** – If this bit is set, then APIC is enabled. If not set, you might want to use `wrmsr` to set it to `0b1`.
- **BSP** – If this bit is set, then the current processor is the one used to start the system (also known as the Bootstrap Processor, or BSP).

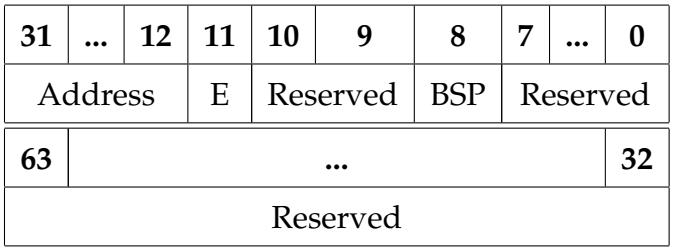



Figure 2.6: MSR result on `IA32_APIC_BASE`

Note that if you have some sort of paging implemented, you need to make sure that the address for APIC corresponds to the one provided by the MSR. Also, to avoid

undefined behavior, you need to ensure that the page that the APIC is mapped on is marked as *strong uncachable*.

A list of all registers present in the APIC specification [1] is presented in Appendix B. In this paper we are going to focus on End Of Interrupt (EOI) and Spurious Interrupt Vector in order to fully set up the APIC device, and later on, when we are going to talk about timers, we are going to tackle Timer Initial Count, Timer Current Count, and Timer Divide configuration registers.

In order to correctly enable the local APIC, we need to set up the spurious interrupt vector. Those interrupts are minor exceptions that appear because of race conditions, incorrect or incomplete acknowledgement, or misdirected/dropped interrupts when working in SMP systems. The spurious interrupt vector has the format presented in Figure 2.7. The focus field is an optional feature named "focus processor checking" that we are not going to tackle in this paper; therefore, it is safe to set it to zero and ignore it for the moment. The Enable flag tells the CPU that the APIC software is enabled (one) or disabled (zero). The vector field indicates the entry in the IDT that has to treat this exception. Therefore, in order to set up the APIC, you need to write to this registry `0x1XX`, where `0x100` enables the APIC and `0xXX` should be the entry in the IDT (for backwards compatibility, it is suggested that this value should be between `0xF0` and `0xFF`).

| **31** | **...** | **10** | **9** | **8** | **7** | **...** | **0** |
|---|---|---|---|---|---|---|---|
| Reserved | | | Focus | Enable | Vector | | |

Figure 2.7: Spurious Interrupt Vector Registry Structure

Finally, as for the PIC, the APIC needs to know that the interrupt was handled correctly and successfully. Therefore, in your interrupt handler, you need to write to clear the End Of Interrupt Register to 0. Writing any other value here may cause undefined behavior.

#### 2.2.2.3 I/O APIC

The I/O APIC's primary function is to receive external interrupt events from connected I/O devices and to redirect them as interrupt messages to xAPIC. The only exception is the LAPIC Timer, which communicates directly with the xAPIC – timers will be discussed in a future chapter. Except for that, any other devices are going to use the

I/O APIC.

The only problem is that the I/O APIC is slightly harder to set up. Firstly, we need to obtain the I/O APIC base from the MADT, and then we are going to need to read the Interrupt Source Override table and initialize the IO Redirection table entries for the interrupts that we want to enable.

Let's start by getting the base address of the I/O APIC. In order to do this, we need to parse the Multiple APIC Description Table (MADT), which is a table available in the Root System Description Table (RSDT), which is found in the Root System Description Pointer (RSDT), which is provided by the bootloader.

Depending on your used bootloader, you may need to make several preparations. Because we are using the Limine crate, obtaining the RSDP address is as simple as making a request. After getting the address, we need to build a data structure similar to the one presented in Figure 2.8. The signature is an 8-byte string that must contain "RSD PTR" and is not null terminated. The checksum is a value that needs to be added to all other bytes in this struct in order for the sum to be zero; if not zero, this struct must be ignored. The field that we are actually interested in is the RSDT address, which represents the physical address of the RSDT data structure. [29]

| Offset | Size | Description |
|---|---|---|
| 0 | 8 | signature |
| 8 | 1 | checksum |
| 9 | 6 | oem id |
| 15 | 1 | revision |
| 16 | 4 | rsdt address |

Figure 2.8: RSDT memory layout

The RSDT is a data structure used in the Advanced Configuration and Power Interface (ACPI) programming interface. In this table, we obtain pointers to the different description tables present in the system. Since this data structure is part of the ACPI, it has a common header with all the other ones (this header is presented in Figure 2.9). The length refers to the length of the entire structure. After the header, we have to read a list of entries; each entry is a 4-byte address to a System Description Table (SDT). Since they are all part of the ACPI, they all have the same header (i.e., the header presented in Figure 2.9); therefore, in order to search for a certain table, we need to look at

its signature, which is a non-null-terminated string. Common entries that you might use are ”MADT” or ”HPET”.

| Offset | Size | Description |
|---|---|---|
| 0 | 4 | signature |
| 4 | 4 | length |
| 8 | 1 | revision |
| 9 | 1 | checksum |
| 10 | 6 | oem id |
| 16 | 8 | oem table id |
| 24 | 4 | oem revision |
| 28 | 4 | creator id |
| 32 | 4 | creator revision |

Figure 2.9: RSDT memory layout

After we have found our MADT entry in the RSDT, we need to parse it. The header is the same as for the other ACPI tables. After the header, there is a four-byte address to the local APIC and a four-byte flag. If bit zero in the flag value is set, that means that you have a Sual 8259 Legacy PIC installed, and you need to make sure to mask all its interrupts in order to avoid undefined behavior (you should probably do this anyway just to be safe). After the address and the flag, there should be a list of variable-length entries. Each entry has one byte that indicates its type and one byte indicating its length. In Appendix C, a list of all entry types is presented.

There are several I/O APIC registers (presented in Figure 2.10). Accessing those registers is done by writing to two memory-mapped addresses:

- **Select** – Located at I/O APIC Base
- **Data** – Located at I/O APIC Base + `0x10`

We are interested in the Redirection Table registry. In order to enable a specific interrupt, as for PIC, we need to give it an index in the IDT by assigning an offset. Firstly, we need to select the corresponding interrupt in the I/O APIC. The selection is done by adding to the Redirection Table Registry address the interrupt index (e.g., 2 for Timer) multiplied by two. After we have written the corresponding value to

the selected memory map address, we need to update the index of the interrupt. For example, let's say that we want to set the timer to 35; then we would write to the data address 32 after we have written to the selected registry.

| Value | Description |
|---|---|
| `0x0` | ID |
| `0x1` | Version |
| `0x2` | Arbitration Priority |
| `0x10` | Redirection Table |

Figure 2.10: I/O APIC registers

# Chapter 3

# Memory Management

When thinking about an operating system and its key goals, one of them that pops to mind is memory management. Managing memory is one of the most important jobs of an operating system and consists of keeping track of which parts of memory are in use, allocating regions for processes when they need it, and deallocating it when they are done [23]. This managing is needed for security. Imagine writing a banking app that manages funds from various clients, but on the same machine, someone is running another app that scans the main memory to keep some logs. We wouldn't want that logger to make copies of our clients data; therefore, we need some protection. We need a mechanism that runs at the system level that knows what process has a certain piece of memory and ensures that it is owned only by him. Here memory management comes into place. On `x86` platforms, there are two different approaches that tackle memory protection: segmentation and paging.

## 3.1 Segmentation

Segmentation is a memory management approach that has been supported since 1978 on some Intel chips; its main goal was to increase the amount of addressable memory. Since in that time, many CPUs only used 16-bit addresses, which translated to 64 KiB of accessible memory, there was a need to increase this limit. Therefore, the CPU started to support additional segment registers such as:

- `cs` – Used for Code Segments
- `ss` – Used for Stack Segments

- `ds` – Used for Data Segments
- `fs` & `gs` – Used Freely

In its first versions, the segment descriptors contained offsets that pointed directly to a certain address; therefore, no access control was performed. This changed later, when CPUs started supporting protected mode. In protected mode, the segment registers didn't contain offsets anymore, but indexes into local or global descriptor tables, which contained, in addition to an offset, the segment size and access permissions. By loading separate global/local descriptor tables for each process, which confined memory accesses to the process's own memory areas, the operating system can isolate processes from each other[19].

### 3.1.1 Virtual Memory

Let's say that we have a program that gets some input from a file, makes some memory transformations, and writes its content to another file. This program has its own memory offset that it can run on. What would happen if, by mistake, we ran two or more instances of this same application at a time? The memory would be corrupted by process `n` writing on process `zero`. Therefore, we need some sort of mechanism that allows a process to have some sort of offset that points to different areas–this is presented in Figure 3.1. On a simplified level, this is the concept behind virtual memory: we need a way to abstract away the memory addresses from physical storage devices. Instead of accessing the memory directly, we should have a translation mechanism first. For segmentation, the translation that is being made is adding an offset to an address. For ease of use, when we are going to talk about physical address, we will refer to the address obtained after the translation, and virtual address refers to the address before the translation.

### 3.1.2 Fragmentation

Now that we have allocated some blocks for two processes in our physical memory, let's try to add another one and see what happens. If we take a look at Figure 3.2a, we can see that we have enough space (200) to allocate another block of size 150. The only problem is that we cannot set an offset without accessing blocks that are owned

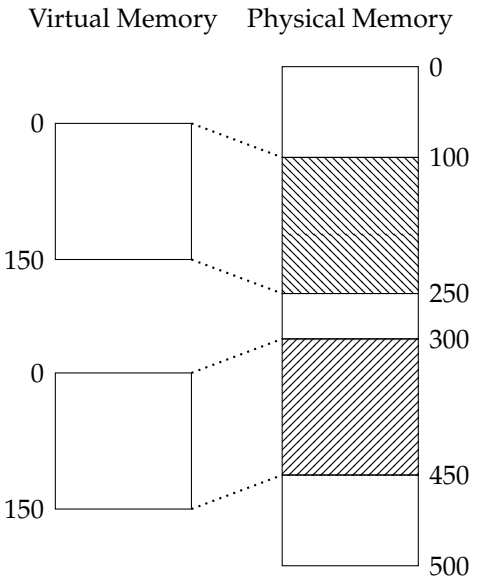
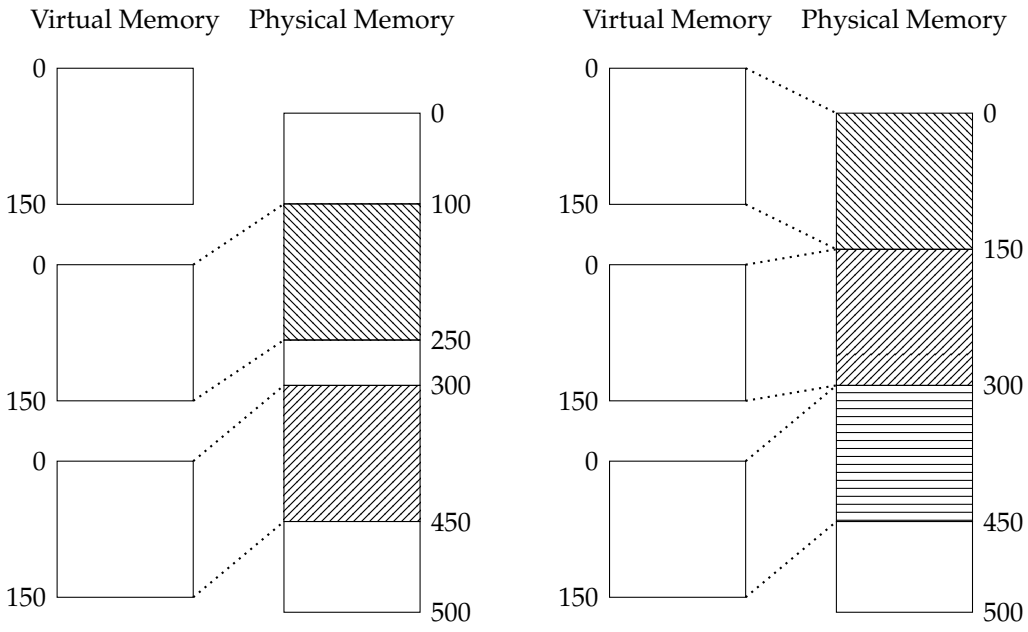



Figure 3.1: Virtual to Physical Memory Mapping with Offsets

by other processes. Therefore, we need to pause all running processes, apply a defragmentation algorithm, allocate the new block, and resume execution. A visual representation of the physical memory after the defragmentation algorithm was applied can be seen in Figure 3.2b. The only problem with this approach resides in its complexity. It is highly inefficient to stop and rearrange memory in order to allocate a new block every time. This is why we currently use paging instead of segmentation.




(a) Before defragmentation

(b) After defragmentation

Figure 3.2: Fragmentation

## 3.2 Paging

The idea behind paging is that we want to avoid as much as possible fragmentation, so let's analyze the problem. We had some processes loaded into memory; those processes were positioned randomly in memory, therefore leaving gaps. What if we assign sections of virtual memory (named pages) to sections of physical memory (named frames) and allocate those pages to the processes that need them? By doing this, we minimize the external fragmentation, and we can allocate memory more efficiently. For example, in Figure 3.3 is presented how the allocation of the third process would be made if it were to use paging instead of segmentation.

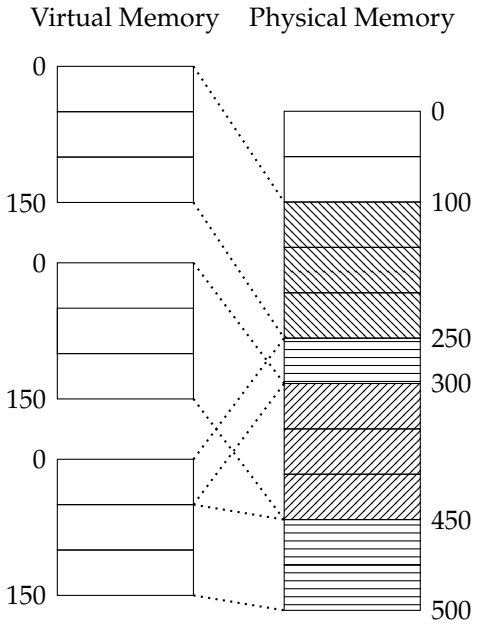



Figure 3.3: Virtual to Physical Memory Mapping using paging

The catch is that paging is not fragmentation-free. For example, if we had a process that needed a block of 101, in our example we would still need 3 pages, therefore wasting 49 bytes. Despite still wasting memory, internal fragmentation is preferred over external fragmentation because it does not require defragmentation and makes the amount of fragmentation predictable (on average half a page per memory region) [19].

Since each individual page needs a connection with a frame, we need some sort of entry to store this information. In Figure 3.4 is presented an example for such an entry, named page table. Keep in mind that we cannot skip fields in this entry.

For example, let's say that we have a process that uses four pages: `0`, `1_000_000`, `1_000_050`, and `1_000_100`. The entries from `50` to `999_950` will be empty in the

| Page | Frame | Flags |
|---|---|---|
| 0 | 100 | r/w |
| 50 | 150 | r/w |
| 100 | 200 | r/w |
| 150 | - | - |

Figure 3.4: Page Table Example

table. This wastes a lot of memory, so we need to find a fix . Let's add another table that contains pairs of memory regions and a level 1 table–the level 1 table contains the page, frame, and flags. If we have this additional table, we go from those `1_000_000` empty entries to just 100 (if table 2 uses regions of size `10_000`). More examples of this technique can be found in Philipp Oppermann's blog on writing an operating system in Rust [19]. Modern architectures use three, four, or more levels of these page tables.

Each page table has a fixed length of 512 entries, each entry has a size of 8 bytes, and therefore each table has a size of $512 \cdot 8B = 4KiB$, fitting exactly into one page on `x86` architecture. In Figure 3.5 is presented a memory layout of the virtual address, where Sign Ext represents a sign extension (i.e., copying the 47th bit), and L# idx represents the index in the Level # table. An example that illustrates better how this translation works can be found in Philipp Oppermann's blog [19].

| **64** | **...** | **48** | **47** | **...** | **39** | **38** | **...** | **30** | **29** | **...** | **21** | **20** | **...** | **12** | **11** | **...** | **0** |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Sign Ext | | | L4 idx | | | L3 idx | | | L2 idx | | | L1 idx | | | Page offset | | |

Figure 3.5: Virtual Address layout

Since most modern operating systems run on 64-bit architectures, many bootloaders start the system directly in long mode. Long mode is an extension of protected mode, which requires paging to be enabled; therefore, when you write your own operating system, you are most likely to already use paging in your system. The reason why we still need to set up paging in our operating system is because the bootloader only sets paging as read-only, so if we try to write to some random address, the CPU will raise a page fault exception.

In order to access the correct page offset, firstly we need to parse the tables, but the tables cannot be accessed because they are located on physical memory, and we only have access to virtual memory. There are several solutions presented in the fol-

lowing sections.

### 3.2.1 Identity Mapping

One simple solution would be to identity map all page tables. This way, every physical address is also a valid virtual address, so we can directly access them.

The only problem with this approach is fragmentation. This method clutters the virtual address space, making it challenging to identify continuous regions of memory where you can allocate large sizes. Consequently, this approach leads to excessive complexity.

### 3.2.2 Fixed Offset Mapping

To avoid cluttering the virtual address space, we can use a separate memory region for page table mappings. So instead of identity mapping them, we can map them at an offset, for example, an offset of 10 TiB.

This approach still has the disadvantage that we need to create a new mapping whenever we create a new page table.

### 3.2.3 Complete Mapping

Mapping the entire physical memory, including page tables and frames of other address spaces, can solve the previously presented problems.

The disadvantage of this method is that additional page tables are needed to store mappings in the physical memory. These page tables need to be stored somewhere, so they can become a problem on low-memory devices.

### 3.2.4 Recursive Page Tables

Another intriguing approach that requires no additional page tables is to map the page table recursively. The idea is to map an entry in the level 4 table to itself, therefore tricking the processor into thinking that it reads a level 3 table. This recursion can be done multiple times to achieve the desired depth.

However, this too can have disadvantages, such as taking large chunks of virtual memory in order to apply the recursion, or that it relies heavily on the `x86` architecture; therefore, there is a possibility that it won't work on other architectures.

More details on this can be found in Philipp Oppermann's blog post about Paging Implementation[20].

### 3.2.5 Implementation Details

In order to start implementing a paging module, we first need to get a physical memory offset from our bootloader. In limine, nothing is easier. Since this bootloader maps our kernel on the higher half of the memory, we need to make a Higher Half Direct Mapping Request (HhdmRequest), which will give us a physical memory offset.

After we have gotten our physical offset, we need to read the contents of the `cr3` registry, which will provide us the active physical frame of the active level 4 table. We then need to add it to the physical offset and use it as a page table. Now you can iterate over this collection of entries and see what pages map to what frames.

Now, in order to convert a virtual address to a physical address, we need to do the following steps:

1. Obtain the level 4 address from `cr3`.
2. Get a list of indexes for all four levels.
3. Convert the virtual address to a page table reference.
4. Use that table to search through available indexes and locate your frame.

Until this point, we only looked at pages as read-only, but what if we want to modify them? In this case we need an allocator that retrieves usable frames. In order to do that, we need to make a request to the bootloader in order to get a map of present memory regions and apply some filters on them. For example, we are only interested in usable frames, so we need to look for this flag and collect them into a list for future use.

A better explanation of this process is illustrated in Philipp Oppermann's blog in reference [20].

## 3.3 Heap Allocation

When writing an application in a programming language, there are usually two types of variables: static variables, which are found on the call stack and are valid only

in the surrounding context, and dynamic variables, which are located on the heap. The heap supports variables that do not have a fixed size and can grow indefinitely. Most heap implementations work by providing the user with two methods: `alloc` and `dealloc`. The desired behavior of those two methods is presented below:

- `alloc` – It accepts a size as its parameter, which can be expanded to include size + alignment, and provides a raw pointer to the initial byte of the memory block allocated. When an error occurs, the alloc should return a null pointer.
- `dealloc` – This function should receive a pointer that points to the memory to be deallocated and does not return any value.

Before digging into our allocator, we first need to initialize a heap that our kernel can use. We can do this by firstly defining a start and a size. Secondly, we need to get a range of pages that contain our heap and finally to mark them as present and writable.

Now that we have initialized our heap, we need to put some thought into developing a suitable allocator. The most important thing that we need to worry about is to not return pointers to already allocated blocks because we can introduce undefined behavior. Therefore, in what follows, we are going to present three allocator designs: Bump, Linked List, and Fixed-Size Block.

### 3.3.1 Bump Allocator

The simplest design is also known as a stack allocator. It allocates memory linearly and only keeps track of how many allocations have been done. The main idea behind its allocation is to bump a pointer on each allocation and to reset it when the allocated counter reaches zero. The behavior of the two necessary functions is explained below.

The allocator starts with two members set to zero: `next` and `allocations`. The `next` member represents a pointer that points to the start of the free memory block and is always bumped towards the end of the heap. The `allocations` member consists of a counter that keeps track of how many regions have been allocated; when this data member reaches zero, then `next` should be set to zero too.

- `alloc` – Increments the `allocations` variable and updates `next` as `next = next + size` where `size` represents the size of the region that has to be allocated

- `dealloc` – Decrements the `allocations` variable, and when it reaches zero, sets `next` to zero.

The main disadvantage of the bump allocator is that it can use deallocated memory only after all allocations have been freed. This means that a single long-lived allocation can prevent our allocator from reusing free memory.

### 3.3.2 Linked List Allocator

The linked list allocator, also known as the pool allocator, takes advantage of a linked list, keeping track of the size of the current node and the next free region. This structure is also commonly referred to as a free list.

This allocator starts with one data member, `head`, which contains a pointer to the top of our list of free blocks. On allocations, we need to truncate or mark a node as being used and rewire the connections in order to keep the list connected. In order for this allocator to work, it is recommended that you use several methods for adding a free region in the front of the list, finding a region that fits a size to be allocated, and returning a pointer to a successfully allocated region.

- `alloc` – Gets the size of a region that should be allocated, searches through the list in order to find a node that has at least the size required, truncates it or removes it in order to mark it in use, and returns the pointer.

- `dealloc` – Adds a free region in the front of the list to mark it as usable. At some point during the execution, you should check whether you have two free blocks in succession in order to merge them.

The main disadvantage of this implementation is that after a certain amount of time, you end up with a fragmented heap of smaller regions, and when trying to allocate a bigger block than your node max size, you get an error as you would not have space for it. Therefore, you need a defragmentation algorithm that should check whether you have two blocks that are both empty and are connecting in order to merge them. This defragmentation is pretty slow for large regions of memory.

### 3.3.3 Fixed-Size Block Allocator

The main idea of this approach is to modify the concept of a linked list in order to save time. Therefore, instead of having a big linked list for the whole stack, we have

multiple linked lists of fixed size. When allocating a memory region, we round up to the next closest size and return a free memory region from that linked list. Because of this optimization, we do not need to go through the entire linked list, and we can return the head of the corresponding list, being sure that it will certainly fit our requirements.

Because of this particularity, if not implemented right, a fixed-size block allocator can waste a lot of memory. For example, let's say that we have the predefined blocks as 16, 32, and 512. If we want to allocate a block of 128, we would need to allocate a block from the 512 list, and we would waste an enormous amount of memory. Therefore, we need to be careful when defining the sizes. This way, you can use powers of two (4, 8, 16, 32, 64, 128, ...) or other common block sizes used when programming, such as 24.

On the other hand, we need to take into account what happens when we need an allocation that is bigger than our predefined sizes. Making big predefined sizes would defeat the purpose and worsen the efficiency. Therefore, it is a good idea to have a fallback allocator that should be called whenever we encounter a size that is bigger than what we have already predefined.

- `alloc` – You should check what the next bigger size is in relation to your request. If it is bigger than what you have defined, you should call the fallback allocator. This usually is a linked list allocator, but any can do. If the size matches one of your predefined sizes, pop the head of the list and return it.

- `dealloc` – When deallocating, check if your size is one of your predefined ones; if so, add it to the front of the corresponding list and update the head pointer. If it does not fall into a predefined size, you should let the fallback allocator do the deallocation.

Some optimizations or variations can be made in order to increase the performance of our allocator. Some popular allocators are the slab allocator and the buddy allocator. The slab allocator is based on the size of predefined types supported in our operating system, therefore optimizing the search space and minimizing the number of calls to other fallback allocators. The buddy allocator is somewhat more interesting. Instead of using a linked list to store free regions, it uses a binary tree. The root contains the entire size of the heap; when allocating, it splits a node in powers of two until a certain degree of fragmentation is reached. When deallocating, it checks its neighbor; if it is also free, the two nodes are merged. The only problem is represented by the internal

fragmentation; therefore, this allocator is usually combined with other allocators, such as the slab allocator.

# Chapter 4

# Multitasking

One of the core components of an operating system is the ability to manage multiple processes running at the same time. When you watch something online or play a game, you probably have other programs running in the background, like music players or communication software. All those have to run at the same time in order to make the user feel as if he can use thousands of applications at the same time on one machine. This effect is called parallelization, and while it can be achieved by using multiple CPUs or cores in a computer – for example, if you have 16 cores, you can run 16 processes in true parallelism – usually, the operating system tries to create an illusion. On most systems, the operating system starts with only one core; after that, it is its job to simulate the parallelism by either starting other cores or implementing some sort of scheduling. This paper will not focus on the process of enabling multiple cores; therefore, we are only going to talk about different types of multitasking.

There are two main forms of multitasking: cooperative and preemptive multitasking. When talking about cooperative multitasking, we think of a system where one process has access to the CPU and no other. When that process finishes its job on the CPU, it needs to give it to the next process. The main disadvantage of this type of multitasking is that you need to trust the process that it will eventually let go of the CPU in order for other processes to use it. If we have an application that might run indefinitely, then we have a problem, because no other processes would have access to the CPU. On the other hand, preemptive multitasking relies on the operating system to give time on the CPU to each process by implementing some sort of scheduling using timers and context switching. More on those topics will be discussed in the following sections.

## 4.1 Cooperative Multitasking

This type of multitasking is based on the idea that the process should voluntarily give up control of the CPU in order for other processes to use it. This step is done either by the programmer or the compiler by inserting some sort of `yield` operation into the program. For example, a `yield` could be inserted after each iteration of a complex loop. It is quite common that when writing some piece of code, we might require data from some source that takes time to deliver, like a database or an HTTP request. In those scenarios, it is better to not occupy the CPU for nothing and let other processes use it.

Since tasks pause themselves, it is not the job of the operating system to save their tasks because the process can save it itself. By doing this, the task switching is much faster than in preemptive cases, because when we use preemptive multitasking, we need to save the whole context of the running process and change it to the context of the next process in order for that one to run correctly. When using cooperative multitasking, the process saves only the state that it needs, the result of a complex operation, for example, without worrying about registers or other things that would normally be saved in preemptive.

The main drawback of this method is represented by uncooperative tasks that can occupy the CPU for an indefinite amount of time, therefore slowing down the system. For this reason, the user of a system should not be trusted to run tasks that are not known to be fully cooperative.

However, despite this drawback, the performance and memory benefit of this method is pretty appealing. Therefore, we are going to dive into implementation details of the cooperative model in programming languages and how we could offer support for them in our operating system. Since this paper is based on an operating system written in Rust, we are going to tackle the Rust async/await model, presenting some particularities and some implementation details.

### 4.1.1 Cooperative mechanism in Rust

The Rust programming language offers support for cooperative multitasking in the form of async/await. But before we can dive into what async/await is and how it works, we need to understand futures and asynchronous programming in Rust.

A future represents a value that is not ready yet. This could be anything, from

common data types to custom ones. Therefore, a future value makes possible the continuation of execution until the requested value is needed.

When working with the async/await model in Rust, the compiler converts the body of the async function into a state machine, where each await call represents a different state. In order to continue its execution, the state machine needs to keep track of its state internally. In addition, it must save all variables that it needs to continue with. This is where the compiler really shines: since it knows which variables are used when, it can automatically generate structs with exactly the variables that are needed.

When working with futures in Rust, we need a poll method to constantly verify the state of our request. This method needs as a parameter a Pin generic over the current object. When we use async/await there are cases when the internal state contains references to itself. Consider a reference to the final element within a vector. When storing the state, such a self-referenced struct might be moved, for example, when called in a function; in this case, the reference can point to a garbage address. In order to avoid this, compilers usually update pointers on the move or store an offset instead of a reference. The Rust compiler, however, forbids moving structs that are self-referenced, thereby preventing undefined behavior. But if not careful, there can be workarounds; therefore, Rust implements a pinning API that prevents certain operations in order to keep the memory safe. Therefore, it prevents moving states in memory between poll calls using futures.

Using futures, we can work asynchronously, but until those futures are polled, nothing happens; therefore, asynchronous code is never executed. With a single future, we can always wait in a loop until it returns our value, but for processes that create a large number of futures, it is highly inefficient to do this. Therefore, we need a global executor that should be responsible for polling all futures in the system until they are finished.

The main purpose of an executor is to allow spawning futures as independent tasks. The executor is then responsible for polling all futures until they are completed. The big advantage of managing all futures in a central place is that the executor can switch to a different future whenever a pending response is returned. There are many designs for executors that can become very complex. The executor that we are going to tackle takes advantage of the waker API in Rust.

Wakers are basically some notifiers that are passed to futures when they are created. When a future is partially completed, the waker sends a signal to the executor,

communicating that it can start pooling the current future because it will soon be ready to return a value.

More details on how Rust implements the async/await model and how futures work can be found in Phillip Opperman's blog referenced in [18].

### 4.1.2 Implementation Details

A very basic executor implementation can use a queue in order to process tasks in a first-in, first-out manner. This executor can use a dummy waker that does nothing. The way that futures would be processed would be in a loop, where the executor repeatedly checks for updates without waiting for any signals. While this method is a proof of concept for how async/await works, it is highly inefficient because it constantly uses the CPU by endlessly checking tasks.

The problem with this simple, constantly checking executor suggests that there must be a smarter, event-driven way to run tasks, especially for operations that involve waiting for hardware in a kernel.

Firstly, we would need some sort of `Task` type that would incorporate our future. This abstraction ensures that a future does not necessarily need to return some data, but it can execute operations, manage other futures, and so on. Having a data structure that represents a task is not sufficient; in addition, we would need some sort of ID that identifies certain tasks in our list. Ideally, when a waker wants to notify the executor that a task is ready, it should be able to provide an ID of its task, and the executor should be able to find the corresponding task as fast as possible. Fast search is not a characteristic of a queue, so maybe we could use something more specific, like a balanced binary search tree map. Most programming languages already provide implementations for this kind of data structure. If your programming language does not have one already implemented, there are plenty of tutorials and papers on this kind of data structure.

When implementing our waker, we need to guarantee that the communication mechanism between the waker and the executor is correctly implemented so it allows async access to it. A mutex would suffice for this purpose, but of course there are other, more language-specific mechanisms, such as `Arc` in Rust, that would serve the purpose. Knowing that we might have multiple futures that might be ready at the same time, we could use a queue for keeping track of finished futures.

In order to save some CPU time, in our executor's `run` method, we should halt the CPU. We would firstly need to disable interrupts in order to avoid undefined behavior. Then check whether any tasks are done running; if not, we can put our CPU in a halt state, which would enable a low-power mode until the next interrupt. Doing so makes sure that we do not waste CPU time.

## 4.2 Preemptive Multitasking

The preemptive multitasking solves the problem of uncooperative tasks. Taking the responsibility to give up the processor from the user and moving it to the operating system. The operating system then needs to have some way to schedule each process for some CPU time. This is achieved using the scheduler, which is a piece of software that is like a hardware multiplexer. The main idea of a scheduler is that it takes many inputs, programs, and threads and allows them to share a single output, which allows them to use the CPU. The working mechanism is pretty simple: an interrupt is fired (usually a timer), the kernel checks whether the current process exceeded its time on the CPU, and if so, it puts it on pause and gives access to the CPU for the next process, and repeats.

### 4.2.1 The Scheduler

The scheduler is the part of the kernel responsible for selecting the next process to run on the CPU, as well as keeping track of what processes or threads exist on the system. The scheduler's method for selecting the next process that can use the CPU is open to debate; many methods exist, ranging from general purpose to specific purpose. In this section, we are going to take a look at the round-robin method, also known as first-come, first-served.

There are some questions that have to be asked: when does the scheduler run, and how long does a thread run until we replace it with the next one? When does the scheduler run? This question is fairly simple to answer: the scheduler usually runs during a timer interrupt. But there are other times when you may find it suitable to run it, for example, during a slow I/O operation that has to complete (e.g., network, disk) or when some programs finish early and want to give up the CPU because they do not use it anymore. As per how long a task should run before it has to be swapped

out, there are many nuances; different systems use different times that a program can run before it has to be paused – named quantum – Some use 10 ms quantums, while others use 100 ms quantums. This decision is heavily debated and can be influenced by anything, up to personal preferences. For ease of use, we are going to use ”thread” and ”process” interchangeably, since for now we are not going to tackle the difference between them. The main part of the scheduler will be thread selection.

- When called, the first thing the scheduler needs to do is determine whether the current thread can be preempted or not. Some kernel routines may disable pre-emption for various reasons, or some processes may run for more than one quantum.
- We must save the context of the current thread in order to resume it later.
- Select the next thread that has to run.
- Optionally, you can make some checks for wakeup timers or for sleeping threads.
- Now we should load the context of the selected thread and mark it as running.

### 4.2.2 Setup

In order for a scheduler to work, it needs a list of processes. The simplest way that this list can be implemented is by using a linked list. This is done because of its dynamic size and ease of access. Some basic methods that you should need for this implementation are `add_process`, `delete_process`, and `get_next_process`.

Once we've created this linked list, we have to decide what fields we need to store some data. Minimally, we would need a context, which represents a snapshot of the CPU in the moment when the scheduler paused the execution, and some sort of status, some flags that can indicate to us that either the process is ready, running, or dead. After choosing a process that can run on the CPU, we need to store some sort of pointer to it in order to have quicker access to it.

Once we have set up our data structures, we need a way to start our scheduler. The best way is to add a function `scheduler()` in our timer interrupt handler that would trigger our scheduler.

There is one more thing that can be discussed: checking for pre-emption. We are not going to tackle this case in our paper, but it is worth mentioning that it is an

interesting problem with interesting solutions. Whether a process needs more time than others suggests some sort of priority that has to be implemented. For now, we are going to switch processes on every timer interrupt because it is easier to implement and understand.

Since we are running our scheduler in an interrupt handler, we can take advantage of the fact that when an interrupt occurs, the CPU pushes the context in which the interrupt happened into the stack; therefore, all we have to do is push our general-purpose registers and change stacks. This process is illustrated below:

1. When a timer interrupt is fired, the CPU pushes the IRET frame on the stack. We then push a copy of what all the general-purpose registers looked like before the interrupt. This environment is the context of the interrupt code.

2. We place the address of the stack pointer into the `rdi` register, as per the calling convention, so it appears as the first argument for our `schedule()` function.

3. After the selection function has returned, we use the value in rax for the new stack value. This position is where a return value is placed according to the calling convention.

4. The context saved on the new stack is loaded, and the new IRET frame is used to return to the stack and code of the new process.

You may want to add an idle process that runs when no other process is running. This ensures that the kernel doesn't just run random pieces of bytes. More information about this topic can be found in the book written by Ivan G. and Dean T. titled Osdev Notes, referenced in [5].

# Chapter 5

# Timers

Timers are useful for all sorts of things, from keeping track of real-world time to triggering scheduling. Many timers are available, including both standard and non-standard types. In this section, we are going to take a look at the main timers for the `x86` architecture.

On a high level, a timer should be able to accomplish the following objectives:

- Should generate interrupts.
- Offer support for periodic mode.
- Should be pollable in order to determine how much time has passed.
- It should have a predefined way of counting.
- Have a certain accuracy, precision, and latency.

First, we must decide what we will use the timer for. Are we going to implement a scheduler? We should choose one that supports interrupts. Do we need some timers to benchmark functions? We would need a timer that supports polling, because it is faster to poll rather than waiting for interrupts. For certain use cases, we need different timers. At the end of this chapter, we will have a section dedicated to analyzing the differences between them.

## 5.1 Calibrating Timers

There are multiple timers that do not have a predefined frequency; those timers usually are based on the frequency of the CPU used. The local APIC timer and the

TimeStamp Counter (TSC) are some excellent examples. In order to use these, we have to know how fast each tick is in real-world time. The easiest way to do this is by using another timer that has a well-known frequency, like PIT or HPET.

The actual calibration process is pretty straightforward and is presented in what follows. For now we will refer to the timer that has a known frequency as the reference and the timer to be calibrated as the target. Those conventions will make it easier to explain the calibration process.

1. Ensure both timers are stopped.
2. If the target timer counts down, set it to the maximum allowed value. If it counts up, set it to zero.
3. Establish a time that we want to calibrate for. This should be long enough that some ticks pass but shouldn't be too long because we risk rolling over one of the timers. A suitable starting place is between 5 and 10 milliseconds.
4. Start both timers and poll the reference timer until the calibration time has passed.
5. Stop both timers and look at how many ticks have passed.
6. Given the calibration time and the amount of ticks that our target counted, we should be able to determine the exact frequency of our target timer.

Keep in mind that if you test your operating system in a virtual machine or on less stable hardware, the results of the calibration might vary. It is useful to do multiple calibrations and compare their results in order to remove the odd ones. It may also be useful to calibrate the timers during the execution of your operating system, therefore correcting small errors over time.

## 5.2 Programmable Interval Timer(PIT)

Although the PIT was first implemented in the original IBM PC, it remained the standard. Since it is quite old, nowadays we do not have a proper PIT but rather an emulation performed by another piece of software, such as HPET.

At its origins, the PIT had other use cases, like providing the clock for the RAM or the oscillator for the speakers. Each of those functions was performed on a different

channel, channel zero being used for the timer and channels one and two having other use cases. On modern PITs, it's most likely that the only channel working is zero, so you might want to leave the others untouched.

In order to work with PIT, we need to do operations on IO ports; those are presented in Figure 5.1. When working with those ports, we firstly need to set up a configuration. The configuration is one byte worth of data, and its structure is presented in Figure 5.2. The BCD/B field tells the PIT whether we want to work with values that are 16-bit binary values or 4-digit BCD values.

| I/O Port | Description |
|---|---|
| `0x40` | Channel 0 data port |
| `0x41` | Channel 1 data port |
| `0x42` | Channel 2 data port |
| `0x43` | Mode/Command register |

Figure 5.1: Ports available for PIT operations

| **7** | **6** | **5** | **4** | **3** | **2** | **1** | **0** |
|---|---|---|---|---|---|---|---|
| Channel | | Access | | Mode | | | BCD/Binary |

Figure 5.2: PIT configuration

The operating mode is selected through the mode bits in the PIT configuration; this mode tells the PIT how it should operate. There are multiple modes available with different applications. What we might be interested in is Mode 2 – Rate generator. All other modes are presented in the PIT-corresponding article on the OsdevWiki blog.

The Access field specifies how the input should be interpreted. Since PIT works with 16-bit registers, we can decide whether the first byte we send is the low or the high byte, or whether we want to use one or the other.

Bits 6 and 7 contain the channel that we want to use. On most modern machines, those should be left at zero.

After we set up the config register, we need to tell PIT where to start the timer. Since it has a base frequency of $3579545/3$ Hz, we can divide this number by $1000$ in order to get how many ticks we have per millisecond and multiply with the number of milliseconds that we want to wait until the timer fires an interrupt. Pay attention

to not overflow the register; writing `0x10000` would set the timer to 0 ticks. Some considerations may be taken when working with APIC, because PIT is a system-wide device. In order for the interrupts to fire correctly, you need to set up I/O APIC to route the interrupt to one of the xAPICs.

## 5.3 High Precision Event Timer(HPET)

The HPET was meant to be the successor for the PIT as a system-wide timer, with more options; however, its design has been plagued with latency issues and occasional glitches. With all that said, it is still more accurate and precise than PIT. However, not all systems support HPET; some disable it via firmware.

In order to check whether your system has an HPET or not, you need to search through the ACPI tables. Iterating through ACPI tables was covered in the subsubsection 2.2.2.3. In addition to what is presented in the corresponding section, we need to build a struct similar to what is presented in Figure 5.3. The HPET specification explains most fields, but we want the address field. The address field represents the physical address where we can obtain all HPET registers.

| Offset | Size | Description |
|---|---|---|
| - | - | ACPI SDT Header |
| 0 | 4 | event timer block id |
| 4 | 4 | reserved |
| 8 | 8 | address |
| 16 | 1 | id |
| 17 | 2 | min clock tick |
| 19 | 1 | page protection |

Figure 5.3: HPET memory layout

The HPET has a single main counter that counts up and some comparators that trigger interrupts whenever certain conditions are met. The HPET will always have at least 3 comparators but can have up to 32. While when working with PIT, we have the frequency stated in its specification, the HPET has its frequency saved in one of its registers. In the table presented in Figure 5.4 is presented a structure that helps us determine the address of a register, starting from the base address–where `R = 0x20`

* N.

| Offset | Register | Type |
|---|---|---|
| 0x0 - 0x7 | General Capabilities and ID Register | Read only |
| 0x10 - 0x17 | General Configuration Register | Read/Write |
| 0x20 - 0x27 | General Interrupt Status Register | Read/Write |
| 0xF0 - 0xF7 | Main Counter Value Register | Read/Write |
| 0x100 + R - 0x107 + R | Timer N Config and Capability Register | Read/Write |
| 0x108 + R - 0x10F + R | Timer N Comparator Value Register | Read/Write |
| 0x110 + R - 0x117 + R | Timer N FSB Interrupt Route Register | Read/Write |

Figure 5.4: HPET registers

We can read the main counter at any time, which is measured in timer ticks. In order to convert these ticks into real time, we need to multiply the number of ticks with the number of femtoseconds that the timer counts for each tick – 1 femtosecond = $1^{-15}$ seconds. In order to get the main counter's period, we need to take a look at the General Capabilities and ID Register presented in Figure 5.5.

| **64** | **...** | **32** | **31** | **...** | **16** | **15** | **14** | **13** | **12** | **...** | **8** | **7** | **...** | **0** |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| period | | | id | | | legacy | reserved | 64-bit enabled | timers | | | revision | | |

Figure 5.5: General Capabilities and ID Register

We are interested in the period field, which would give us the time in femtoseconds. Other intriguing fields are "legacy," which tells us whether the HPET emulates PIT and RTC or not, and "64-bit enabled," which, if set, enables us to work with timers on 64-bit.

In order to start working with our timer, we need to set bit 0 in the General Configuration Register to 1, which will start the timer. Keep in mind that you need to enable interrupts for your timer if you want to use it via an interrupt handler. Furthermore, we need to set bit 1 to value 0 in order to disable the emulation mechanism for PIT and RTC.

The timer itself does not generate interrupts. In HPET, the comparators are responsible for this task. In order for the HPET to properly emulate PIT and RTC, it uses the first two comparators to generate interrupts for them. All comparators support

one-shot mode; whether one supports periodic mode can be checked in the Configuration and Capability Register of each comparator. Below in Figure 5.6 is presented the memory layout of such a register.

| **64** | **...** | **32** | **31** | **...** | **16** | **15** | **14** | **13** | **...** | **9** | **8** | **...** | **0** |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| IntMap | | | reserved | | | FSBCnf | | IntRout | | | Config | | |

Figure 5.6: Configuration and Capability Register

The IntMap field indicates a bitmap where if bit X is set, then the comparator can be mapped to the IRQX line of your interrupt controller. For FSBCnf, if bit 1 is set, then this timer supports FSB interrupt mapping, and if bit 0 is set, it will use the corresponding mapping. In IntRout is the actual routing to your interrupt controller. The config setup is presented in Figure 5.7.

| Bit | Description |
|---|---|
| 8 | For 64-bit timer, if this field is set, the timer will be forced to work in 23-bit mode. |
| 7 | This bit is reserved for future use, should not be used. |
| 6 | This field is used to allow software to directly set periodic timer's accumulator. |
| 5 | This is a read-only field. If this is set, then the timer is in 64-bit mode, otherwise is in 32-bit. |
| 4 | This is a read-only field. If this is set, the timer supports periodic mode. |
| 3 | This bit selects periodic mode (1) or one-shot mode (0). If bit 4 is set to 0, this field will always be set to 0. |
| 2 | If this bit is set to 1, it enables triggering of interrupts. |
| 1 | If 0, this timer generates edge-triggered interrupts, otherwise it generates level-triggered. More on this in the OSDev article on Generic Interrupt Controller[28] |
| 0 | This bit is reserved for future use, should not be used. |

Figure 5.7: HPET registers

We have presented many details in this section. Below we will present a summarized version of what you need to do in order to set up HPET:

1. Locate the HPET base in the ACPI tables.
2. Find and calculate HPET frequency.
3. Set up comparators for interrupt triggering if necessary.
4. Start the timer.

## 5.4 Local APIC Timer

The local APIC timer is a hardwired component of the CPU; this way you do not have to do any resource management in order for it to work properly. Each processor, or core, gets its own Local APIC Timer, so if you need multiple timers to do different things, you can enable and set up multiprocessor support and use Local APIC Timer (we will use LAPIC Timer for this section to refer to Local APIC Timer). The main downside is that it does not have a fixed frequency. It oscillates at one of the CPU's frequencies. In order to make use of it, you firstly need to calibrate it.

In order to use the LAPIC Timer, you should firstly set up the xAPIC and I/O APIC; this process is presented in the chapter 2. After we have set up the xAPIC and I/O APIC, we only need three registers:

- `0x3E0` – Divide configuration, this helps us increase the range of our space. For example, if we want to track how long something takes, but that value would be higher than 64-bit, we would use a divider.
- `0x320` – Timer entry in the LVT, this is where we tell APIC what index in the IDT the timer has.
- `0x380` – The initial count of the timer, here we set up where the timer starts.

## 5.5 Comparations

So far we have covered three different timers. Depending on our objectives, we could use one or the other. In the table presented in Figure 5.8, there are some comparisons made regarding features.

When working with each counter, we need to contemplate what kind of trade-offs we are considering. If we want higher accuracy and relatively fast set-up, we might

| Timer | Interrupts | Periodic | Polling | Counting | Frequency |
|---|---|---|---|---|---|
| PIT | Yes | Yes | No | Down | 1.193182 MHz |
| HPET | Yes | Yes | Yes | Up | 14.318 MHz |
| LAPIC | Yes | Yes | Yes | Down | needs calibration |

Figure 5.8: Comparation between different timers

choose HPET. If we do not need high accuracy and we are only interested in how fast we can obtain a timer up and running, we might use PIT. But if we need multiple timers for tasks such as scheduling, we might consider working with LAPIC Timer.

Since we are writing a general-purpose operating system, we might need several functions for calibration, sleeping, and pooling. After reliably implementing a timer, we might consider adding some of those functionalities for an easier workflow.

# Chapter 6

# File Systems

All computer applications need to store and retrieve information. While a process is running, it can store a limited amount of information within its own address space. However, the storage capacity is restricted to the size of the virtual address space. For some applications this size is adequate, but for others, such as airline reservations, banking, or corporate record keeping, it is far too small. Other problems that may occur are related to the persistence of information; for example, a database should keep its contents intact even after some random blackouts, or the accessibility from different processes. [23]

Most modern operating systems offer solutions to those problems by implementing some sort of file system. The core purpose of a file system is to keep track of the used/free space and to maintain information in a fairly organized way.

There are many different file systems out there, some proprietary and some not, some very basic and some feature-rich. When implementing an operating system, you need to establish what you will use it for. Would you operate on critical information that cannot be lost? Maybe you need a file system that focuses on data protection. Do you only need some sort of interface that lets you write information on disk? You should definitely choose some easier option.

There are multiple file systems that you may choose to implement. There are file systems that need comprehensive understandings of their inner workings, such as NTFS or BTRFS. An advanced file system may seem good, but it's better to start with a simpler one and build from there. Below we will present some common "beginner" file systems with their pros and cons.

- **USTAR**

+ The easiest to implement
+ Supports special files (devices, symlinks)
+ Supports Unix Permissions

- Not a file system in the common understanding of the term. Most commonly used for compression.
- Generally read-only
- No standard partition type for it
- No support for fragmentation

- **RAMFS/TMPFS**

  + High flexibility of implementation
  + Fast

  - Not persistent

- **FAT**

  + Common. It can be read or written by all OSes
  + Relatively easy to implement.

  - There is no support for files larger than 4GB.
  - No support for Unix permissions

- **EXT2**

  + Supports large files
  + Supports Unix permissions
  + Can be used by most modern operating systems

  - Out of ”beginner” file systems, this is the most complex.

- **BMFS**

  + Supports large files
  + Highly documented

- No support for fragmentation
- Less control over source code

If none of the above are appealing to you, you could, of course, implement your file system. Writing a good file system can be in itself a pretty good project, but since it is not the objective of this paper, we are not going to talk about the process of writing your file system.

We will discuss implementing the EXT2 file system (also known as ext2fs), considering its pros and cons, as it is fairly popular. In order to do this, we would need some things set up. If you use a bootloader that offers support for file systems, you may use information provided by it. Since limine does not support any file system, you are left to parse it and implement it from scratch.

Before going into developing the ext2fs, we need some way to communicate with the disk. This is where the disk driver comes into place. We prefer to concentrate on the development of the file system instead of a complex driver for a communication protocol with the physical storage device. Therefore, we are going to implement a simple ATA IDE driver.

If you use the Q35 machine in QEMU to run your OS, you may have trouble with ATA IDE. Because Q35 does not support this standard, you have to either switch to the PC machine or implement another disk communication driver.

## 6.1 ATA IDE

ATA IDE, commonly referred to as ATA PIO, is a device driver that has as its core objective providing a safe and user-friendly interface for disk operations. This ATA driver is a beginner-friendly approach because it is easy to implement, and according to the ATA specification, all ATA-compliant drivers should support this mode.

Since ATA PIO relies on the CPU's IO port bus and not on the memory to write commands, this operation mode is extremely slow. It can achieve speeds up to 16 MB/sec but that is in extremely rare cases that do not allow any other processes to run during those operations.

Because of its hardware design, there is only one wire that can select which drive is selected in order to perform a certain operation. Because of this limitation, there can be only two operational devices on any ATA bus. Those devices are usually called

*Master* and *Slave*. There is no superiority between those two drives; their names are only a convention.

However, most chips support two ATA buses per chip, therefore calling them primary and secondary. Adding those two, we can connect up to four storage devices on our system. Both primary and secondary drives can be operated on by using some registers or by using interrupts (using interrupts is highly discouraged because it can use more CPU cycles). In Figure 6.1 are presented the necessary ports and interrupt indexes that we need in order to continue.

| Bus | IO Port Base | Control Port Base | IRQ |
|---|---|---|---|
| Primary | `0x1F0` | `0x3F6` | 14 |
| Secondary | `0x170` | `0x376` | 15 |

Figure 6.1: Primary and Secondary information

According to the ATA specs, in order to send any ATA command, you firstly need to check the BSY and DRQ flags (those are explained below) in the status register. Since IO ports are slow, we need to give the device some time to put the correct data in our response. According to the OSDev Wiki, you should examine the status register fifteen times and only pay attention to the last value returned. [27]. Since it can be assumed that an IO operation takes 30 nanoseconds before the status is ready, you would need to wait around 400 - 450 nanoseconds. In order to overcome this, you would need to either have a function that does dummy operations on ports or to have implemented a sleep function with a timer. You need to keep in mind that any operations when DRQ, BSY, or ERR flags are set are prohibited and can lead to undefined behavior.

Working with ports, the ATA specification has multiple commands that can be used. Those are presented in the official ATA specification, referred to in [8]. We are not going to cover the entire ATA8 command set. Instead, we are going to use the commands presented in Figure 6.2.

### 6.1.1 Detection and Initialization

There are several methods of detecting and initializing an ATA drive. One of the easiest is called floating bus. Although not the most reliable, it is a simple way to determine whether an ATA drive is present on your system. The most reliable method, which is also standardized across all BIOSes, is the IDENTIFY command. Before diving

| Hex | Name |
|---|---|
| 0x20 | Read |
| 0x30 | Write |
| 0xA0 | Master Drive Selector |
| 0xB0 | Slave Drive Selector |
| 0xE7 | Cache Flush |
| 0xEC | Identify |

Figure 6.2: ATA Commands

into the detection methods, you should first take a look at the available registers in the ATA specification. Those are presented in Figure 6.3. We denoted IO as the IO base and CTRL as the control base.

| Port | Name | Access |
|---|---|---|
| IO + 0 | Data Register | Read/Write |
| IO + 1 | Error Register | Read |
| IO + 1 | Features Register | Write |
| IO + 2 | Sector Count Register | Read/Write |
| IO + 3 | LBA0 Register | Read/Write |
| IO + 4 | LBA1 Register | Read/Write |
| IO + 5 | LBA2 Register | Read/Write |
| IO + 6 | Drive Register | Read/Write |
| IO + 7 | Status Register | Read |
| IO + 7 | Command Register | Write |
| CTRL + 0 | Alternate Status Register | Read |
| CTRL + 0 | Control Register | Write |
| CTRL + 1 | Drive Blockess Register | Read |

Figure 6.3: ATA Registers

#### 6.1.1.1 Floating Bus

When a disk is selected by the BIOS during the boot time, it is supposed to maintain control of the electrical value for each ATA bus that it is connected to. If there is

no disc connected, all values on the bus will all go to high, which software would read as `0xFF`; this condition is called a floating bus.

Because of this condition, you can easily determine whether a device is connected or not. In order to check this, you have to read the regular status byte **before** any data is sent to the IO ports. The value `0xFF` is an illegal value; therefore, no drive should be connected on that port.

Measuring the float value is a useful shortcut for detecting non-existent drivers, but if the read value is different than `0xFF`, this does not mean that there is an ATA drive or even a drive at all. Therefore, it is highly recommended to do the IDENTIFY verification anyway.

#### 6.1.1.2 IDENTIFY command

The IDENTIFY command is a more reliable alternative to the floating bus verification. In order to complete the identify action, you need to follow the steps presented below. Keep in mind that after each command you need to wait at least 400 ns in order to avoid undefined behavior. The necessary commands and registers are presented in Figure 6.2 and Figure 6.3.

1. Select the target drive by sending the corresponding command to the drive register.
2. Set LBA0, LBA1, and LBA2 to zero.
3. Send the IDENTIFY command to the command register.
4. Read the status register (or the alternative status register).
5. Check whether the value is zero; if so, then the drive does not exist and the process should be terminated. If the status value is nonzero, poll the status register until bit 7 (BSY) is clear.
6. Check whether the LBA0, LBA1, and LBA2 are nonzero. If so, then the drive is not an ATA drive, and the process should be interrupted. If LBA0, LBA1, and LBA2 are zero, you need to continue polling until bit 8 (DRQ) or 0 (ERR) sets.
7. If the ERR is clear, the drive is ready to be operated on.
8. Using the data register, read 256 pairs of u16 and interpret them accordingly.

Useful interpretations of the read bytes are related to LBA28 and LBA48. Pairs 60 and 61, interpreted as a u32, contain the total number of 28-bit LBA addressable sectors on the drive. Pairs 100 to 103, interpreted as a u64, contain the total number of 48-bit addressable sectors on the drive. If any of those numbers are zero, then the corresponding addressing is not available.

In order to get the actual sector size of your device, you need to check bit 10 in word 83; if it is set, your device supports block sizes greater than 512. After checking bit 10, you need to verify words 106-107 and 117-118, which will give you the logical and, respectively, the physical sector size of your device.

### 6.1.2 Accessing the disk

When working with an ATA driver, you are usually making operations on blocks. At this step you need to know how big your sector size is. When reading and writing, you need to ensure that the buffer that you use does not exceed the sector size. In the following sections we are going to describe a 28-bit PIO operation because it is simpler. If you are looking forward to implementing 48-bit PIO, you can check out the OSDev Wiki post on ATA PIO [27].

#### 6.1.2.1 Read

When reading from your storage device, there are certain steps that need to be done before getting any actual data. Those steps are explained in what follows. Make sure that after each command, a delay of 400 ns is present in order for the drive to operate correctly.

1. Build the drive select value by making a logical OR between the drive ID (`0xE0` for master or `0xF0` for slave) and bits 24-28 of the block you are trying to read from.

2. Set the sector count register to 1. In order to make reads of only one sector. May be changed depending on needs.

3. Break down the block into three bytes. The low byte goes into LBA0, the mid byte goes into LBA1, and the high byte goes to LBA2.

4. Send the command to read and wait until the BSY bit is cleared.

5. Now you can read $\frac{block\ size}{2}$ words (16-bit values) from the data register.

It might be useful to write several abstractions for different block sizes, such as 1 KB or 4 KB, in order to ease the development process of the file system.

#### 6.1.2.2 Write

The write process is fairly similar to the read process. The only difference is that you need to send a write command instead of a read one. When writing the write function, you need to make sure that after each portion where you write something to the disk, you flush the content too.

## 6.2 ext2fs

The first Linux release used the MINIX 1 file system and was limited by the short file names and 64 MB file sizes. Eventually, the MINIX 1 file system was replaced by the first extended file system, ext, which permitted both longer file names and larger file sizes. Due to its performance inefficiencies, ext was replaced by its successor, ext2, which is still in widespread use [23].

The second extended file system organizes data on a partition, dividing it into block groups. Each block group contains blocks used to store data. When parsing an ext2fs, the first thing that we have to do is to parse the superblock. The superblock is a critical structure that holds information about the current state of the file system. For redundancy, the ext2fs stores additional copies of important data in each superblock, such that if anything gets corrupted, it can be safely restored.

Each block group will contain a group descriptor table that contains various information about free and occupied blocks or inodes, bitmaps, and other information. The inode is the core component of ext2fs. Each piece of data is represented through an inode. For example, each file, directory, or symlink has its corresponding inode.

The ext2fs uses multiple data structures in order to store important information, such as the superblock, block group descriptor tables, blocks, and inodes. In this section we are going to look closer at those data structures.

### 6.2.1 Blocks and Block Groups

Any kind of storage, from partitions to disks or block devices formatted with the ext2fs, is divided into small groups called blocks. These blocks are grouped into larger units named Block Groups. These blocks help the file system to reduce fragmentation and to speed up operations.

The size of a block is usually determined in the moment of formatting. Different block sizes have different impacts on how the file system behaves. In Figure 6.4 is presented a short analysis of how different block sizes impact the behavior of the file system.

| Upper Limit | 1KiB | 2 KiB | 4 KiB | 8 Kib |
|---|---|---|---|---|
| file system blocks | 2.1e+9 | 2.1e+9 | 2.1e+9 | 2.1e+9 |
| blocks per block group | 8.1e+3 | 1.6e+4 | 3.2e+4 | 6.5e+4 |
| inodes per block group | 8.1e+3 | 1.6e+4 | 3.2e+4 | 6.5e+4 |
| bytes per block group | 8 MiB | 32 MiB | 128 MiB | 512 MiB |
| file system size | 4 TiB | 8 TiB | 16 TiB | 32 TiB |
| blocks per file | 1.6e+7 | 1.3e+8 | 1.0e+9 | 8.5e+9 |
| file size | 16 GiB | 256 GiB | 2 TiB | 2 TiB |

Figure 6.4: Impact of Block Sizes[21]

When parsing the file system, we will stumble upon the Block Group Descriptor Table; we will refer to it as BGDT. The BGDT is an array of block groups used to define parameters of all block groups. It provides the location for the inode bitmap and table, block bitmap, number of free inodes and blocks, and some other useful information.

The BGDT is usually located immediately after the Superblock–on 1 KiB block size, this would be the third block. Depending on how many block groups there are, the BGDT can spread across multiple blocks. When in doubt, refer to the Superblock in order to get relevant information.

The layout of a block group descriptor is presented in Figure 6.5.

The bitmaps are memory regions that keep track of what blocks or inodes are in use or free. When iterating through a bitmap, you need to keep in mind that each bit represents a position. For example, if byte two in this bitmap were `0x1`, the result would mean that the block or inode with number 16 is used, and those from 9 to 15 are free to use.

| Offset | Size | Description |
|---|---|---|
| 0 | 4 | Block Bitmap |
| 4 | 4 | Inode Bitmap |
| 8 | 4 | Inode Table |
| 12 | 2 | Free Blocks Count |
| 14 | 2 | Free Inodes Count |
| 16 | 2 | Used Dirs Count |
| 18 | 2 | Padding |
| 20 | 12 | Reserved |

Figure 6.5: Block Group Descriptor Structure

The inode table points to the first block where this table is located. The inode table structure will be discussed in the following section.

### 6.2.2 Inodes

According to the Linux Kernel Documentation: *The inode (index node) is a fundamental concept in the ext2 file system. Each object in the file system is represented by an inode. The inode structure contains pointers to the file system blocks that contain the data held in the object and all of the metadata about an object except its name.*

When working with objects in an ext2fs, you will need to identify and parse some data structures related to a certain inode. Those data structures are called inode tables, and a memory layout is presented in section D.1.

The most important fields in this structure are presented below.

#### 6.2.2.1 Field: Mode

The mode field sets up the type and the permissions for the corresponding object. It consists of four bytes: `0xGHII`. Where `G` denotes the file format, `H` the process execution, and `II` the access rights. Those are presented in subsection D.1.1.

#### 6.2.2.2 Field: Size

In revision 0, this is a signed 32-bit value indicating the file size. In revision 1 and later, and only for regular files, this represents the lower 32 bits of the size; the upper

32 bits are located in the Dir Acl field.

#### 6.2.2.3 Field: Blocks

Inodes use four levels of block pointers. The direct block pointer points to a block that contains actual data. The indirect block pointer points to a block that contains a list of direct block pointers. A list of indirect block pointers is accessed through the doubly indirect block pointer. The triply pointer leads to a list of doubly indirect block pointers. This syntax can be understood better by taking a look at Figure 6.6, which is a visual representation of this explanation from Wikipedia.

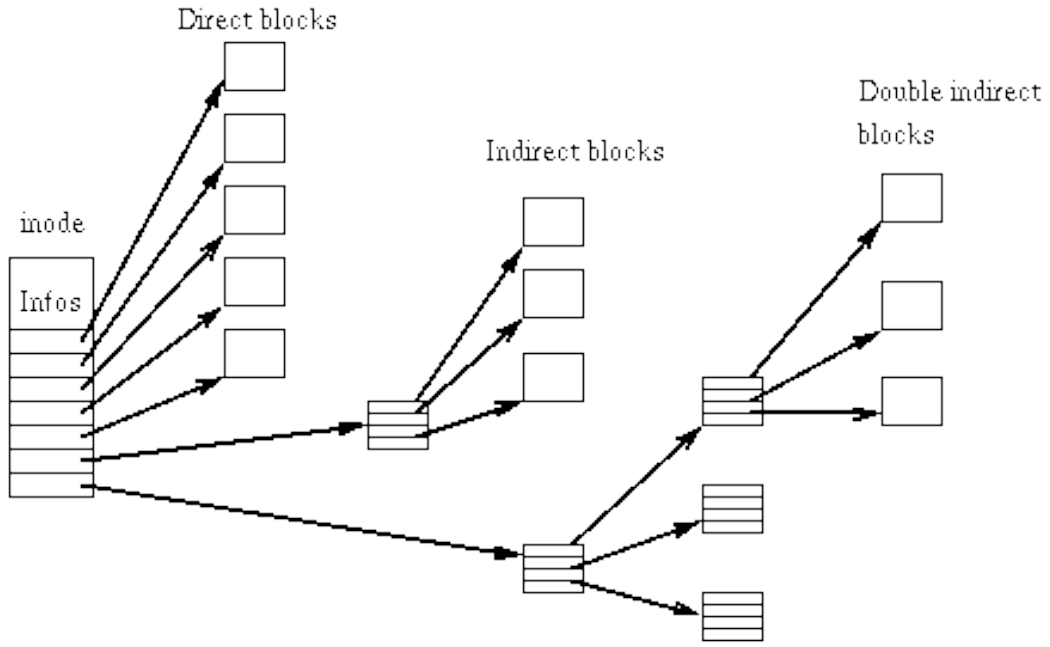



Figure 6.6: Inode block pointers

When parsing block pointers, if your implementation does not support sparse files, when you encounter a zero, you should stop parsing and assume that all other blocks are zero. The zero block is reserved for the bootloader and should not be accessed.

#### 6.2.2.4 Locating an Inode

Inodes are stored in an ascending numerical order. To find a certain inode in your file system, you need to do the following calculations:

$$block\ group = \frac{inode - 1}{inodes\ per\ group}$$

Keep in mind that inodes are numbered starting at 1, so that is why we need to subtract 1 in our formula. The $inodes\ per\ group$ is a value obtained from the superblock.

Once we know which group an inode resides in, we can look up the actual inode by first retrieving that block group's inode table's starting address. The index of our inode in this block group's inode table can be calculated by following [25]:

$$index = (inode - 1)\ MOD\ inodes\ per\ group$$

Now, to determine which block contains our inode, we need to do the following:

$$containing\ block = \frac{index \cdot inode\ size}{block\ size}$$

Where both $inode\ size$ and $block\ size$ are defined in the superblock.

### 6.2.3 Superblocks

The superblock contains all the information about the configuration of the file system. The primary copy of the file system is located at offset 1024 from the start of the device. Since it is so important, multiple copies are located in block groups throughout the file system.

The memory layout of the superblock is presented in section D.2, and some important fields are presented in what follows.

#### 6.2.3.1 Field: Log Block Size

The block size is computed using this 32-bit value as the number of bits to shift left the value 1024. This value may only be non-negative.

An interesting thing is that in Linux, there cannot be a block size bigger than the current page size, so you may want to implement this behavior in order to avoid errors.

#### 6.2.3.2 Field: Magic

This is a magic number that has to be checked in order to ensure that the current disk image is formatted using the ext2fs formatter. The value you should expect is `0xEF53`. If this value is different, you need to stop using the current disk as an ext2 file system.

#### 6.2.3.3 Features

When implementing the ext2fs, the formatter leaves all features in a default state as unsupported. When you work on your implementation, depending on what you implement, you can indicate that your current file system supports certain features, such as sparse files or compression.

### 6.2.4 Directories

When talking about directories in an ext2fs implementation, we are not referring to the common directories that we use in our day-to-day life. A directory in ext2fs represents some type of object that contains certain information, such as the inode to which it corresponds or the name of the object. When parsing a directory, we expect some sort of list of the objects it contains. There are currently two implementations of storing entries in a directory: linked list and indexed. The indexed variant is a newer implementation; it offers backward compatibility for linked lists because it is up to the operating system to implement it. Since we have not worked with optional features, implementing an indexed directory format may be left as an exercise. In what follows, we are going to focus on the linked list format.

In a linked list directory format, each entry contains the name of the entry, the inode associated with its data, and the distance within the directory file to the next entry. In Figure 6.7 is presented the memory layout of the Directory Entry.

| Offset | Size | Description |
|---|---|---|
| 0 | 4 | inode |
| 4 | 2 | rec len |
| 6 | 1 | name len |
| 7 | 1 | file type |
| 8 | 0-255 | name |

Figure 6.7: Linked Directory Entry Structure

This entry contains some useful fields. For example, we can use the inode to read a file or to further parse a directory. The name is an ISO-Latin-1 string (this is the standard expected on most systems). The name len field represents the length of the name and should not be larger than rec len - 8.

The rec len is a field that stores the displacement from the start of the current block to the next entry. The directory entries must be aligned to 4 bytes, and there cannot be any entries spanning multiple blocks; therefore, if an entry does not fit in the current block, you need to update the previous entry to point to the end of the block and move the current entry to a new block. To delete an entry, create an empty entry that points to the next.

The file type is not the same as for inodes; therefore, it is presented in Figure 6.8.

| Value | Description |
|---|---|
| 0 | Unknown |
| 1 | Regular File |
| 2 | Directory File |
| 3 | Character Device |
| 4 | Block Device |
| 5 | Buffer File |
| 6 | Socket File |
| 7 | Symbolic Link |

Figure 6.8: Directory Entry File Types

### 6.2.5 Implementaion Advices

When working on your implementation for your ext2 file system, you may find it hard working with indirect blocks; therefore, it is suggested that you implement some sort of iterator over them. Therefore, you will not write the same code multiple times just to search for some specific information.

In order to read files from the disk, you will need to parse the directory entries recursively from inode 2, which is the root, until you find your desired file. You will also need to ensure that you do not mix your disk driver's (such as ATA) block size with your file system's block size. You will need to implement some translation functions that can ensure fluid communication between those two implementations.

Writing to files will be troublesome until you manage to write a block allocator, for obvious reasons. When encountering this, the iterator that we discussed earlier would be useful.

After you have your allocator, a deallocator should be pretty straightforward to implement. When you have those two components, creating and removing entries from your file system should not be incredibly challenging. If you find yourself in any trouble, you can check the documentation made by Dave Poirier on the second extended file system referenced by [21].

# Conclusions

This thesis has systematically examined the fundamental concepts required to develop an operating system from the ground up. We have investigated core architectural components–including interrupt handling, memory management, process scheduling, file systems, and graphical rendering–that form the backbone of any functional OS. While these elements represent essential building blocks, they merely introduce the vast landscape of operating system design, where every subsystem presents unique technical challenges and opportunities for innovation.

The world of operating systems is boundless, and no single project can capture its full depth. For many developers, this immensity is precisely what makes OS development so captivating. It is a modern-day David-and-Goliath struggle: a lone programmer, armed with nothing but determination and a compiler, standing against the towering complexity of hardware specifications, undocumented behaviors, and emergent bugs. Success demands perseverance–scouring decades-old manuals, wrestling with obscure hardware quirks, and devising creative solutions to problems that few have ever encountered.

Building an operating system from scratch is a monumental task, one that not every attempt survives. Many ventures stall halfway, their architects forced to concede to the sheer weight of the challenge. Yet, even incomplete efforts contribute to a greater collective progress. Every developer who dares to tackle an OS–whether they ultimately succeed or not–leaves behind valuable insights, clearer documentation, or simply the inspiration for others to follow. Each line of code, each debugged issue, and each shared lesson pushes the field forward, making the path slightly smoother for those who come next.

This thesis stands as both a guide and a testament to that enduring effort. By consolidating knowledge on key technologies–from Rust's safety guarantees in driver development to the design of schedulers and file systems–we aim to lower the barrier

for future explorers. The road remains long, but with each step, the once-daunting Goliath of OS development becomes a little more conquerable.

# Chapter 7

# Acknowledgments

Unexpected obstacles can arise during the operating system development process, such as obscure hardware flaws, unclear specifications, or the intricacy of low-level systems programming. I used a variety of tools to get past these obstacles during this project, including AI assistants, which were subtle but useful.

Writing thorough, understandable documentation for classes and methods was one of the most time-consuming parts of this project. Even though I had a thorough understanding of their internal operations, the documentation's repetition frequently took attention away from core development. AI solutions such as GitHub Copilot were extremely helpful in this case, speeding up the procedure without sacrificing accuracy. I did, however, use AI sparingly and purposefully because I genuinely think that relying too much on these technologies stifles real learning. To ensure that all important design choices and problem-solving techniques remained my own, I instead set them aside for specialized, non-creative tasks like documenting code, explaining current implementations, or paraphrasing technical explanations.

Nevertheless, I purposefully refrained from utilizing AI to decipher hardware specs or clarify fundamental ideas. I learned priceless lessons about hardware characteristics, past design decisions, and the "why" behind many standards from going through hundreds of pages of manuals. Human cooperation became crucial at this point. I am incredibly grateful to Mitreanu Alexandru, whose knowledge and direction enabled me to avoid pitfalls and improve my comprehension. No tool was as transformative as their insights and the collective knowledge of the OS development community.

This combination of human mentoring and targeted technical support – a balance

I hope future developers will find for themselves – is what will make operating systems more approachable if this thesis is successful.

# Appendix A

# Exceptions

| Name | Indexes | Type | Error Code |
|---|---|---|---|
| Division Error | 0x0 | Fault | No |
| Debug | 0x1 | Fault/Trap | No |
| NMI | 0x2 | Interrupt | No |
| Breakpoint | 0x3 | Trap | No |
| Overflow | 0x4 | Trap | No |
| Bound Range Exceeded | 0x5 | Fault | No |
| Invalid Opcode | 0x6 | Fault | No |
| Device Not Available | 0x7 | Fault | No |
| Double Fault | 0x8 | Abort | Yes |
| Invalid TSS | 0xA | Fault | Yes |
| Segment Not Present | 0xB | Fault | Yes |
| Stack-Segment Fault | 0xC | Fault | Yes |
| General Protection Fault | 0xD | Fault | Yes |
| Page Fault | 0xE | Fault | Yes |
| x87 Floating Point Exception | 0x10 | Fault | No |
| Alignment Check | 0x11 | Fault | Yes |
| Machine Check | 0x12 | Abort | No |
| SIMD Floating-Point Exception | 0x13 | Fault | No |
| Virtualization Exception | 0x14 | Fault | No |
| Control Protection Exception | 0x15 | Fault | Yes |
| Hypervisor Injection Exception | 0x1C | Fault | No |

| | | | |
|---|---|---|---|
| VMM Communication Exception | 0x1D | Fault | Yes |
| Security Exception | 0x1E | Fault | Yes |
| Triple Fault | -- | -- | -- |

# Appendix B

# xAPIC Register

| Offset | Name | Reset |
|---|---|---|
| 0x20 | APIC ID Register | 0x??000000 |
| 0x30 | APIC Version Register | 0x80??0010 |
| 0x80 | Task Priority Register | 0x00000000 |
| 0x90 | Arbitration Priority Register | 0x00000000 |
| 0xA0 | Processor Priority Register | 0x00000000 |
| 0xB0 | End of Interrupt Register | – |
| 0xC0 | Remote Read Register | 0x00000000 |
| 0xD0 | Logical Destination Register | 0x00000000 |
| 0xE0 | Destination Format Register | 0xFFFFFFFF |
| 0xF0 | Spurious Interrupt Vector Register | 0x000000FF |
| 0x100 – 0x170 | In Service Register | 0x00000000 |
| 0x180 – 0x1F0 | Trigger Mode Register | 0x00000000 |
| 0x200 – 0x270 | Interrupt Request Register | 0x00000000 |
| 0x280 | Error Status Register | 0x00000000 |
| 0x300 | Interrupt Command Register Low | 0x00000000 |
| 0x310 | Interrupt Command Register High | 0x00000000 |
| 0x320 | Timer Local Vector Table Entry | 0x00010000 |
| 0x330 | Thermal Local Vector Table Entry | 0x00010000 |
| 0x340 | Performance Counter Local Vector Table Entry | 0x00010000 |
| 0x350 | Local Interrupt 0 Vector Table Entry | 0x00010000 |
| 0x360 | Local Interrupt 1 Vector Table Entry | 0x00010000 |

| | | |
|---|---|---|
| 0x370 | Error Vector Table Entry | 0x00000000 |
| 0x380 | Timer Initial Count Register | 0x00000000 |
| 0x390 | Timer Current Count Register | 0x00000000 |
| 0x3E0 | Timer Divide Configuration Register | 0x00000000 |
| 0x400 | Extended APIC Feature Register | 0x00040007 |
| 0x410 | Extended APIC Control Register | 0x00000000 |
| 0x420 | Specific End of Interrupt Register | – |
| 0x480 – 0x4F0 | Interrupt enable Register | 0x00000000 |
| 0x500 – 0x530 | Extended Interrupt Local Vector Table Register | 0x00000000 |

# Appendix C

# MADT Entries

## C.1 Type 0: Processor Local APIC

This type represents a single processor and its local interrupt controller.

| Offset | Size | Description |
|---|---|---|
| 2 | 1 | ACPI Processor ID |
| 3 | 1 | APIC ID |
| 4 | 4 | Flags |

If bit zero is set, the CPU can be enabled. If it is not, then you need to check bit one. If this one is disabled as well, the system should not try to enable this CPU.

## C.2 Type 1: I/O APIC

| Offset | Size | Description |
|---|---|---|
| 2 | 1 | I/O APIC's ID |
| 3 | 1 | reserved |
| 4 | 4 | I/O APIC address |
| 8 | 4 | Global System Interrupt Base |

## C.3 Type 2: I/O APIC Interrupt Source Override

| Offset | Size | Description |
|---|---|---|

| | | |
|---|---|---|
| 2 | 1 | Bus Source |
| 3 | 1 | IRQ Source |
| 4 | 4 | Global System Interrupt |
| 8 | 2 | Flags |

## C.4 Type 3: I/O APIC NMI Source

| Offset | Size | Description |
|---|---|---|
| 2 | 1 | NMI Source |
| 3 | 1 | reserved |
| 4 | 2 | Flags |
| 6 | 4 | Global System Interrupt |

## C.5 Type 4: Local APIC NMI

| Offset | Size | Description |
|---|---|---|
| 2 | 1 | ACPI Processor ID |
| 3 | 2 | Flags |
| 5 | 1 | LINT |

## C.6 Type 5: Local APIC Address Override

| Offset | Size | Description |
|---|---|---|
| 2 | 2 | reserved |
| 5 | 1 | 64-bit physical address |

## C.7 Type 9: Processor Local x2APIC

| Offset | Size | Description |
|---|---|---|
| 2 | 2 | reserved |

| | | |
|---|---|---|
| 4 | 4 | Processor's local x2APIC ID |
| 8 | 4 | Flags |
| 12 | 4 | ACPI ID |

# Appendix D

# EXT2FS

## D.1 Inode Table

| Offset | Size | Description |
|---|---|---|
| 0 | 2 | Mode |
| 2 | 2 | Uid |
| 4 | 4 | Size |
| 8 | 4 | ATime |
| 12 | 4 | CTime |
| 16 | 4 | MTime |
| 20 | 4 | DTime |
| 24 | 2 | Gid |
| 26 | 2 | Links Count |
| 28 | 4 | Blocks |
| 32 | 4 | Flags |
| 36 | 4 | Osd1 |
| 40 | 12 * 4 | Direct Block Pointer |
| 88 | 4 | Singly Indirect Block Pointer |
| 92 | 4 | Doubly Indirect Block Pointer |
| 96 | 4 | Triply Indirect Block Pointer |
| 100 | 4 | Generation |
| 104 | 4 | File Acl |
| 108 | 4 | Dir Acl |

| | | |
|---|---|---|
| 112 | 4 | Faddr |
| 116 | 12 | Osd2 |

### D.1.1 Mode Values

| File Format | |
|---|---|
| Value | Description |
| 0xC000 | Socket |
| 0xB000 | SymLink |
| 0x8000 | Regular File |
| 0x6000 | Block Device |
| 0x4000 | Directory |
| 0x2000 | Character Device |
| 0x1000 | Fifo |
| Execution Ovveride | |
| 0x0800 | Ser Process User ID |
| 0x0400 | Ser Process Group ID |
| 0x0200 | Sticky bit |
| Access Rights | |
| 0x0100 | user read |
| 0x0080 | user write |
| 0x0040 | user execute |
| 0x0020 | group read |
| 0x0010 | group write |
| 0x0008 | group execute |
| 0x0004 | others read |
| 0x0002 | others write |
| 0x0001 | others execute |

## D.2 Superblock

| Offset | Size | Description |
|---|---|---|
| 0 | 4 | inodes count |

| | | |
|---|---|---|
| 4 | 4 | blocks count |
| 8 | 4 | r blocks count |
| 12 | 4 | free blocks count |
| 16 | 4 | free inodes count |
| 20 | 4 | first data block |
| 24 | 4 | log block size |
| 28 | 4 | log frag size |
| 32 | 4 | blocks per group |
| 36 | 4 | frags per group |
| 40 | 4 | inodes per group |
| 44 | 4 | mtime |
| 48 | 4 | wtime |
| 52 | 2 | mnt count |
| 54 | 2 | max mnt count |
| 56 | 2 | magic |
| 58 | 2 | state |
| 60 | 2 | errors |
| 62 | 2 | minor rev level |
| 64 | 4 | lastcheck |
| 68 | 4 | checkinterval |
| 72 | 4 | creator os |
| 76 | 4 | rev level |
| 80 | 2 | def resuid |
| 82 | 2 | def resgid |
| Extended Fields | | |
| 84 | 4 | first ino |
| 88 | 2 | inode size |
| 90 | 2 | block group nr |
| 92 | 4 | feature compat |
| 96 | 4 | feature incompat |
| 100 | 4 | feature ro compat |
| 104 | 16 | uuid |

| | | |
|---|---|---|
| 120 | 16 | volume name |
| 136 | 64 | last mounted |
| 200 | 4 | algo bitmap |

# Bibliography


[1] AMD. System programming. *AMD64 Architecture Programmer's Manual*, 2, 2024.

[2] Andries E. Brouwer. Font-formats recognized by the linux kbd package, 2001.

[3] Arthur C. Clarke. *Profiles of the Future*. 1962.

[4] Anderson D. *SATA Storage Technology*. Mindshare Inc., 2007.

[5] Ivan G. and Dean T. Osdev notes. GitHub, 2025.

[6] GeeksForGeeks. Vga full form, 2021.

[7] IMB. Cp00437, 1984.

[8] American National Standard Institute. Ata8-acs. In Curtis E. Stevens, editor, *Information Technology*, 2006.

[9] Intel. System programming guide. *Intel® 64 and IA-32 Architectures Software Developer's Manual*, 3A(1), 2016.

[10] Intermation. Ep 021: Utf-8 encoding examples. YouTube, 2020.

[11] Sebastian Lague. Coding adventure: Rendering text. YouTube, 2024.

[12] Lenovo. What is a vga monitor?, 2025.

[13] Limine. Framebuffer. Docs.Rust, 2025.

[14] Limine. The limine boot protocol. GitHub, 2025.

[15] Philipp Oppermann. Catching exceptions. *Writing an OS in Rust*, 1, 2018.

[16] Philipp Oppermann. Cpu exceptions. *Writing an OS in Rust*, 2, 2018.

[17] Philipp Oppermann. Hardware interrupts. *Writing an OS in Rust*, 2, 2018.

[18] Philipp Oppermann. Async/await. *Writing an OS in Rust*, 2, 2019.

[19] Philipp Oppermann. Introduction to paging. *Writing an OS in Rust*, 2, 2019.

[20] Philipp Oppermann. Paging implementation. *Writing an OS in Rust*, 2, 2019.

[21] Dave Poirier. The second extended file system: Internal layout, 2019.

[22] Rust. Primitive type char. Documentaion, 2025.

[23] Andrew S. Tanenbaum and Herbert Bos. *Modern Operating Systems – Fourth Edition*. Pearson Education Inc., 2014.

[24] VoxelRifts. A brief look at text rendering. YouTube, 2024.

[25] OsDev Wiki. Ext2, 2022.

[26] OsDev Wiki. Apic, 2023.

[27] OsDev Wiki. Ata pio, 2023.

[28] OsDev Wiki. Generic interrupt controller, 2023.

[29] OsDev Wiki. Rsdp, 2023.

[30] OsDev Wiki. Cpuid, 2024.

[31] OsDev Wiki. Drawing in a linear framebuffer, 2025.